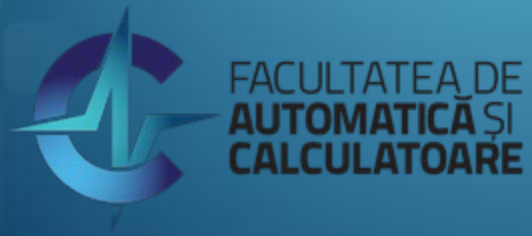


Algoritmi Paraleli și Distribuiți
Introducere

Radu-Ioan Ciobanu
radu.ciobanu@upb.ro





Despre mine





https://mobylabupb.com

radu.ciobanu@upb.ro

https://www.facebook.com/mobylabUPB

Despre curs

- Fizic (miercurea 16-18, vinerea-par 16-18)
- Open-office: lunea 12-18, PR 606
- Teams + Moodle (meta-curs)
- Regulament afișat pe pagina cursului
  - Forma de evaluare a examenului final poate fi decisă/schimbată de către titularul de curs, în funcție de un context specific. Forma de evaluare poate fi: scris sau oral.
- Punctajele obținute pe parcurs sau examen se pot păstra pentru un an universitar (nu acumulare)
- Temele se pot trimite doar pe parcursul primului semestru universitar

Despre curs

- Nota:

  10 = **6** (Parcurs) + **4** (Examen) + **0.5** (Bonus)

  Parcurs **>= 3** Examen **>= 2**

  Parcurs: 3 Teme (**4p**) + Lab (**1p**) + Test lab (**1p**)

  Bonus – teste la curs (**0.5p**)

Întrebări?

Organizare

Sapt	Luna	L	Ma	Mi	J	V	S	D	Laborator	Teme	Curs	Curs practic
1	Octombrie	3	4	5	6	7	8	9			C00 - Introducere	
2		10	11	12	13	14	15	16	Lab 1 (Pthreads)		C01 - Introducere in algoritmi paraleli	P01 - Pthreads
3		17	18	19	20	21	22	23	Lab 2 (Pthreads)		C02 - Complexitate	
4		24	25	26	27	28	29	30	Lab 3 (Pthreads)	Tema 1 (29.10 - 20.11) NEFINAL	C03 - Semafoare	P02 - Java
5	Noiembrie	31	1	2	3	4	5	6	Lab 4 (Java)		C04 - Solutii paralele	
6		7	8	9	10	11	12	13	Lab 5 (Java)		C05 - Abordari LogN	P03 - Sincronizare
7		14	15	16	17	18	19	20	Lab 6 (Java)		C06 - Comunicare prin mesaje	
8		21	22	23	24	25	26	27	Lab 7 (Java)	Tema 2 (26.11 - 18.12) NEFINAL	C07 - Ceasuri logice	P04 - MPI
9		28	29	30	1	2	3	4				
10	Decembrie	5	6	7	8	9	10	11	Lab 8 (MPI)		C08 - Algoritmi unda	P05 - FT
11		12	13	14	15	16	17	18	Lab 9 (MPI)		C09 - Alegere lider	
12		19	20	21	22	23	24	25	Lab 10 (MPI)	Tema 3 (17.12 - 15.01) NEFINAL	C10 - Stabilirea topologiei	P06 - CAP
		26	27	28	29	30	31	1	Vacanta de iarna		Vacanta de iarna	
	Ianuarie	2	3	4	5	6	7	8				
13		9	10	11	12	13	14	15	Lab 11 (MPI)		C11 - Terminarea programelor	
14		16	17	18	19	20	21	22	Test practic		C12 - Toleranta la defecte	P07 - P2P

Bibliografia

G.R.Andrews
Concurrent Programming. Principles and Practice

C.A.R. Hoare
Communicating Sequential Processes

S.G. Akl
The Design and Analysis of Parallel Algorithms

G.R. Andrews, R.A. Olsson
The SR Programming Language. Concurrency in Practice

Ian Foster
Designing and Building Parallel Programs

A.S. Tanenbaum
Structured Computer Organization (Fourth Edition)

Allen B. Downey
The Little Book of Semaphores

Wan Fokkink
Distributed Algorithms: An Intuitive Approach (2nd edition)

Nancy A. Lynch
Distributed Algorithms

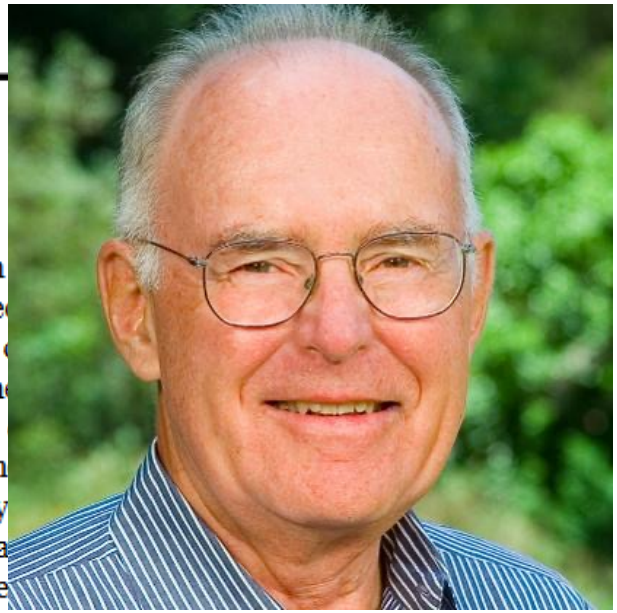
De ce calcul paralel și distribuit?

- Timp de execuție mai scurt
- Permite abordarea problemelor de dimensiuni mari
- Accesul resurselor aflate la distanță
- Reducerea costurilor
- Toleranță la defecte
- Ascunderea timpilor de așteptare
- Redundanța
- Scalabilitatea
- Scăderea timpului de răspuns
- Securitate

Limitele programării secvențiale?

Cramming More Components onto Integrated Circuits

GORDON E. MOORE, LIFE FELLOW, IEEE



With unit cost falling as the number of components per circuit rises, by 1975 economics may dictate squeezing as many as 65 000 components on a single silicon chip.

The future of integrated electronics is the future of electronics itself. The advantages of integration will bring about a proliferation of electronics, pushing this science into many new areas.

Integrated circuits will lead to such wonders as home computers—or at least terminals connected to a central computer—automatic controls for automobiles, and personal portable communications equipment. The electronic wristwatch needs only a display to be feasible today.

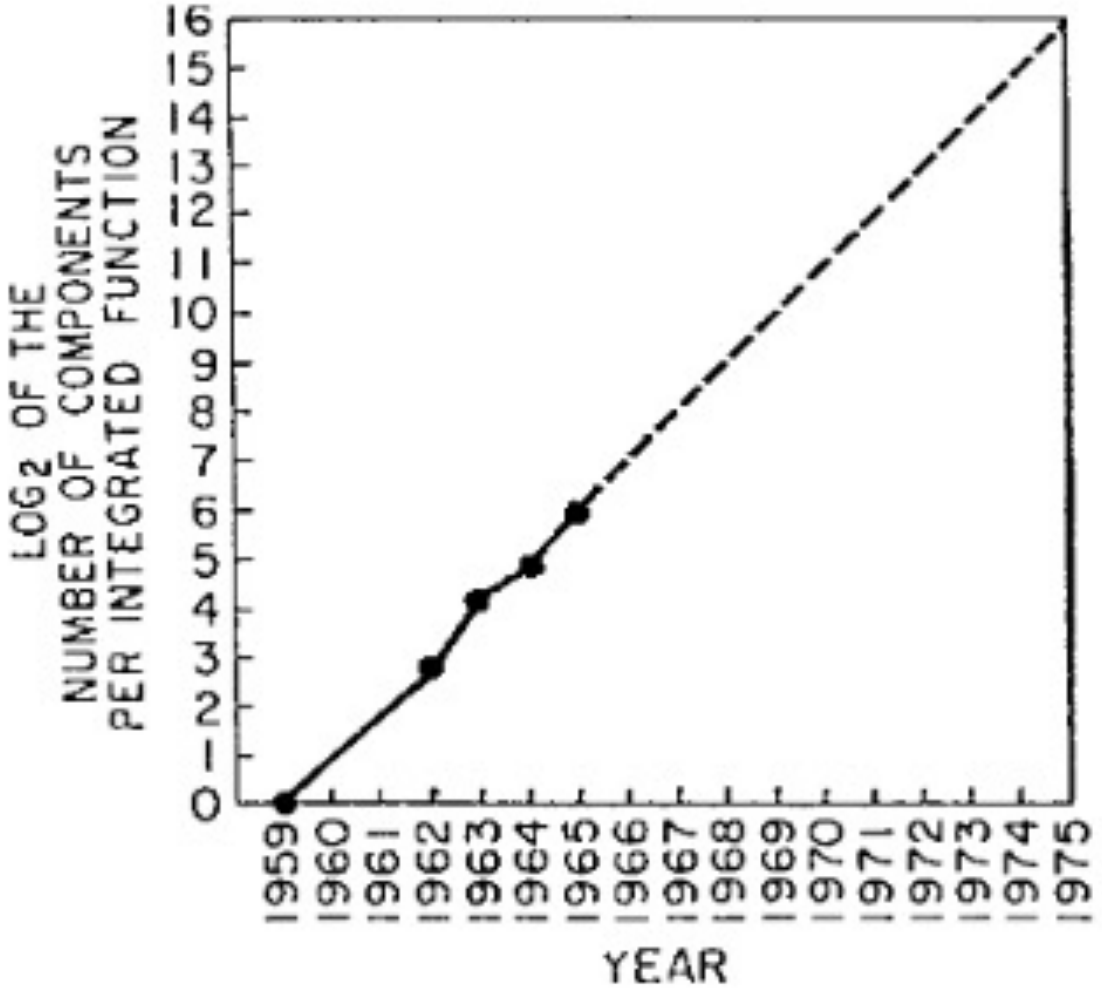
But the biggest potential lies in the production of large systems. In telephone communications, integrated circuits

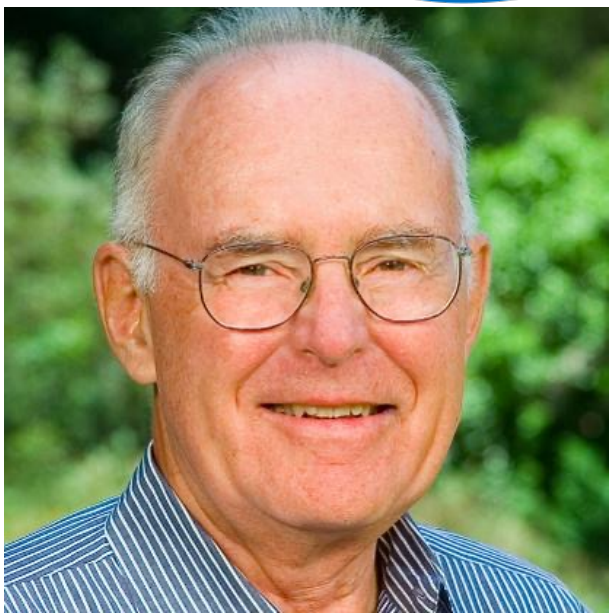
Each approach
each borrowed te
believe the way
various approach

The advocates
already using th
resistors by apply
conductor substra
upon films are de
attachment of active semiconductor devices to the passive film arrays.

Both approaches have worked well and are being used in equipment today.

Limitele programării secvențiale?





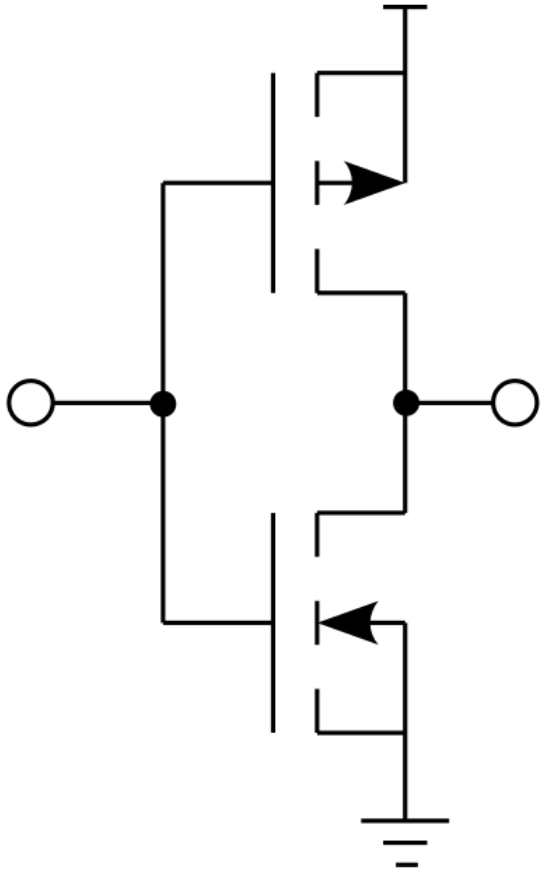
cofounded Intel

Limitele programării secvențiale

- Viteza de transmisie
  - Maxim c – viteza luminii
- Miniaturizare
  - Tranzistor de mărimea unui atom
- Economic
  - Costuri enorme pentru cercetare și proiectarea unui nou tip de procesor

Limitele programării secvențiale

- CMOS



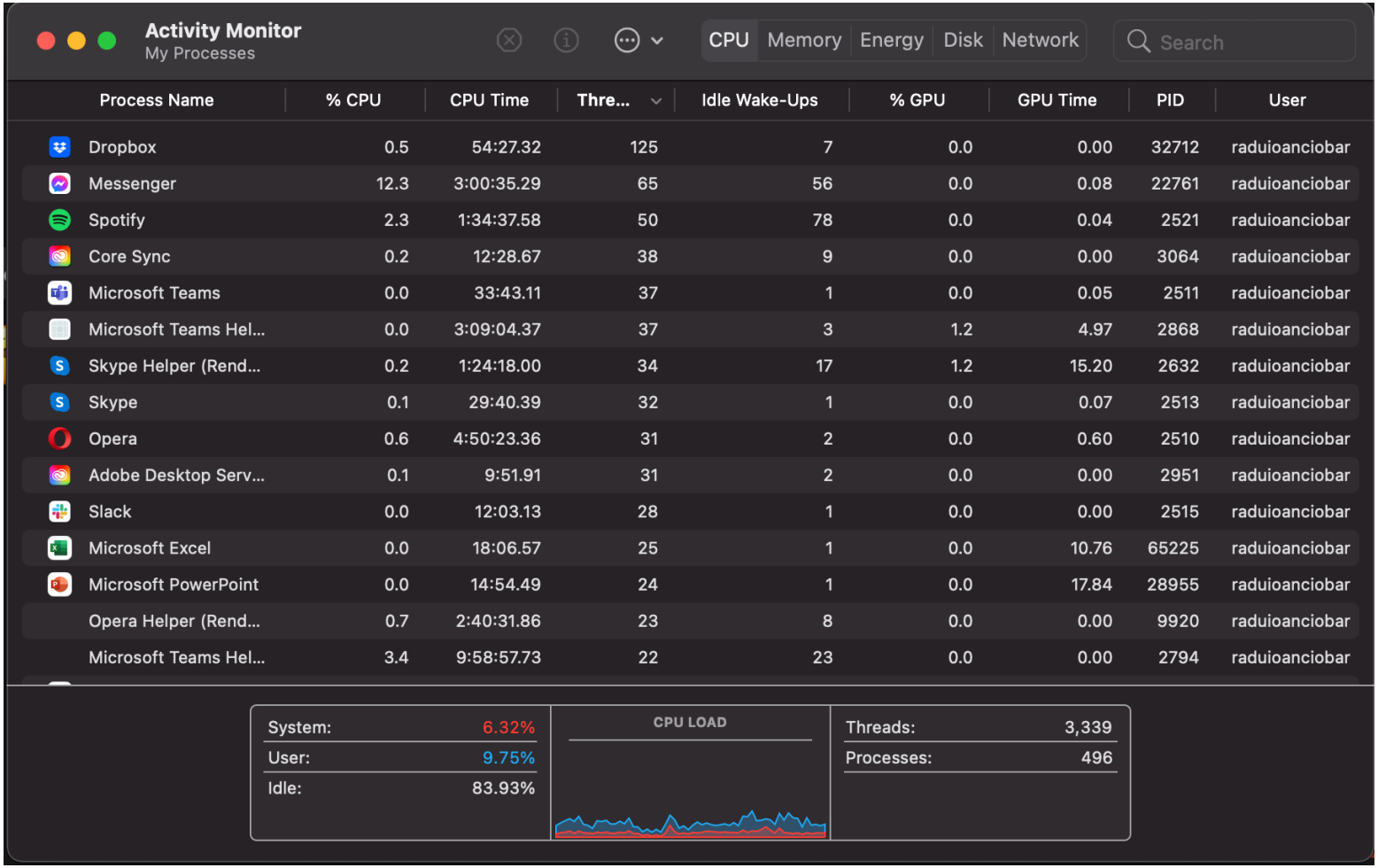
Importanța cursului

- Apar tot mai multe tehnologii paralele și distribuite
  - Blockchain; Peer-to-Peer
- Chiar și un procesor de ceas are mai multe core-uri
  - LG Watch Sport - MSM8909w Processor
    - Quad-Core
- Suport în noile IDE-uri
  - Visual Studio, IntelliJ IDEA
- Număr mare de aplicații distribuite
  - Dropbox; Spark; Boinc

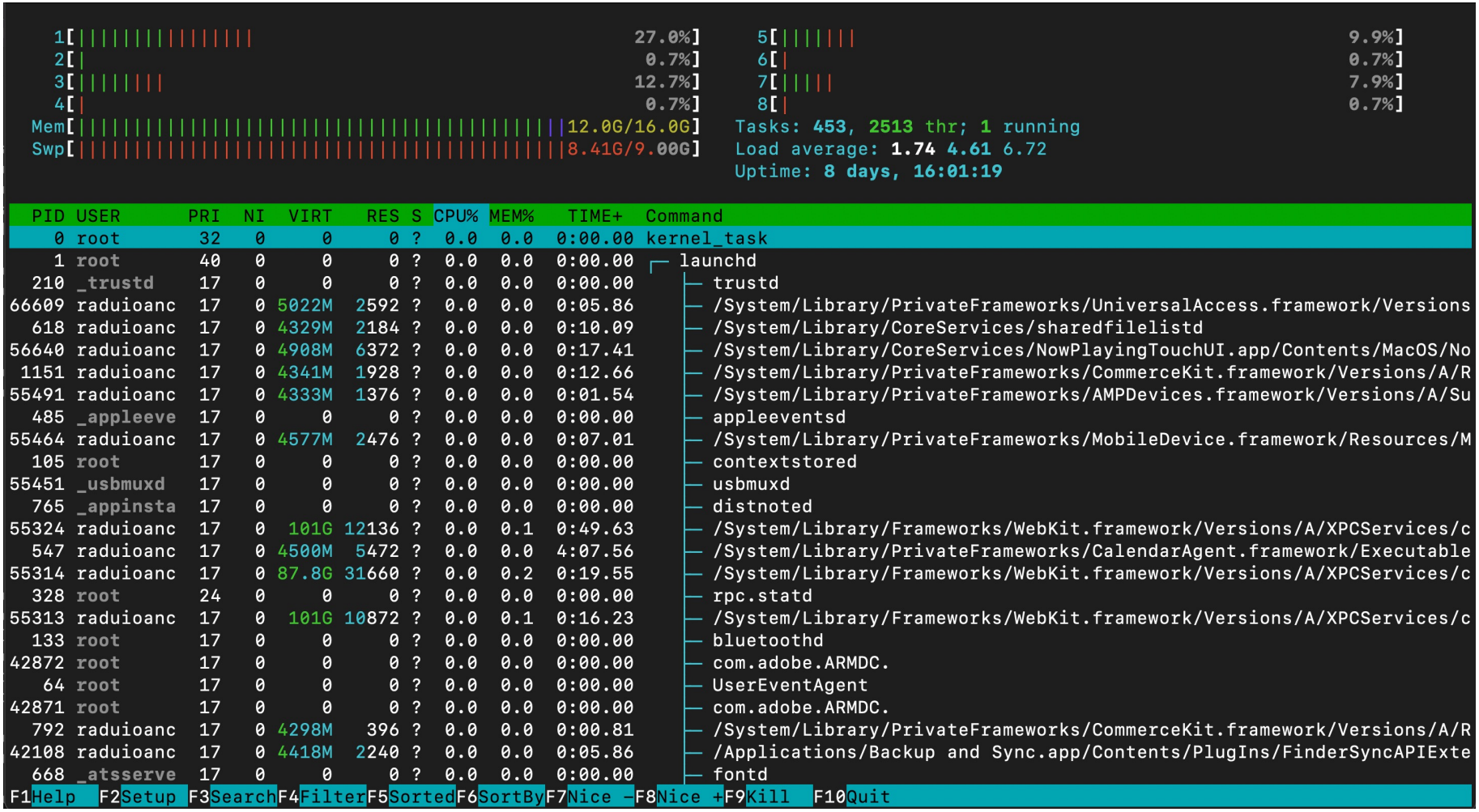


Importanța cursului

- Aproape toate aplicațiile folosesc mai multe thread-uri



Importanța cursului



Supercomputers

Rank	System	Cores	Rmax (PFlop/s)	Rpeak (PFlop/s)	Power (kW)
1	**Frontier** - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE DOE/SC/Oak Ridge National Laboratory United States	8,730,112	1,102.00	1,685.65	21,100
2	**Supercomputer Fugaku** - Supercomputer Fugaku, A64FX 48C 2.2GHz, Tofu interconnect D, Fujitsu RIKEN Center for Computational Science Japan	7,630,848	442.01	537.21	29,899
3	**LUMI** - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE EuroHPC/CSC Finland	1,110,144	151.90	214.35	2,942
4	**Summit** - IBM Power System AC922, IBM POWER9 22C 3.07GHz, NVIDIA Volta GV100, Dual-rail Mellanox EDR Infiniband, IBM DOE/SC/Oak Ridge National Laboratory United States	2,414,592	148.60	200.79	10,096
5	**Sierra** - IBM Power System AC922, IBM POWER9 22C 3.1GHz, NVIDIA Volta GV100, Dual-rail Mellanox EDR Infiniband, IBM / NVIDIA / Mellanox DOE/NNSA/LLNL United States	1,572,480	94.64	125.71	7,438

Frontier


FRONTIER
OAK RIDGE
National Laboratory
U.S. DEPARTMENT OF
ENERGY
Hewlett Packard
Enterprise
AMD

Fugaku



LUMI



Summit



Sierra





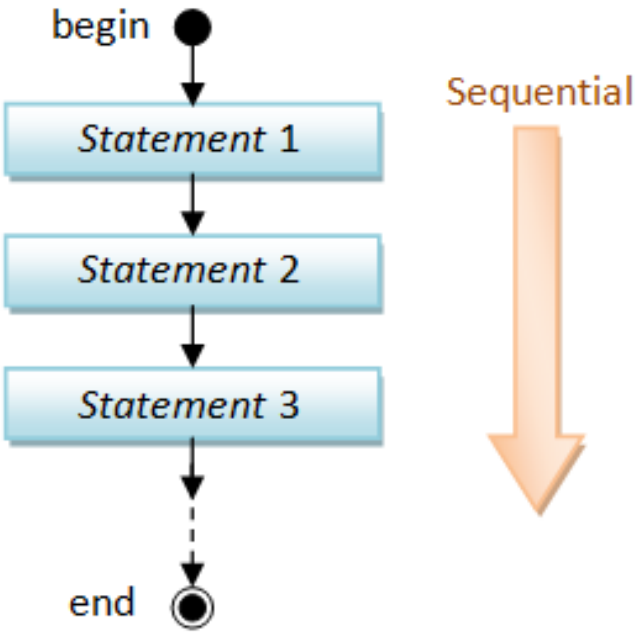
Computing power 2018

24-hour average: 28.019 PetaFLOPS.
Active: 156,141 volunteers, 648,324 computers.

Computing power 2020

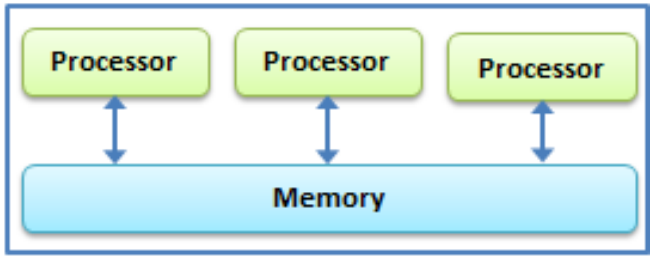
24-hour average: 26.392 PetaFLOPS.
Active: 87,025 volunteers, 826,092 computers.
Daily change: +43 volunteers, 0 computers.

Algoritmi Paraleli/Distribuiți vs Secvențiali

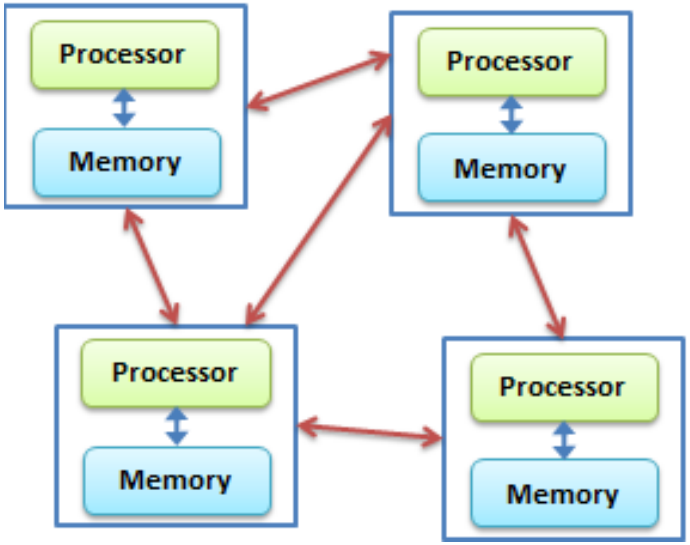


Sequential

Parallel Computing



Distributed Computing



Resurse fizice

- Procesor – multi-core

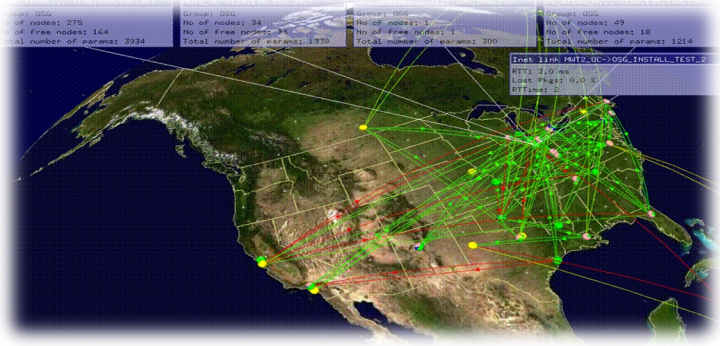




High End Desktop (HEDT)	MSRP / Retail	Cores / Threads	Base / Boost GHz	L3 Cache	Memory
Threadripper 3990X	$3,990 / $3,750	64 / 128	2.9 / 4.3	256	Quad DDR4-3200
Intel W-3175X	$2,999 / N/A	28 / 56	3.1 / 4.8	38.5	Six-Channel DDR4-2666



- Cluster



- Grid/Cloud

Taxonomia Flynn

Some Computer Organizations and Their Effectiveness

MICHAEL J. FLYNN, MEMBER, IEEE

Abstract—A hierarchical model of computer organizations is developed, based on a tree model using request/service type resources as nodes. Two aspects of the model are distinguished: logical and physical.

General parallel- or multiple-stream organizations are examined as to type and effectiveness—especially regarding intrinsic logical difficulties.

The overlapped simplex processor (SISD) is limited by data dependencies. Branching has a particularly degenerative effect.

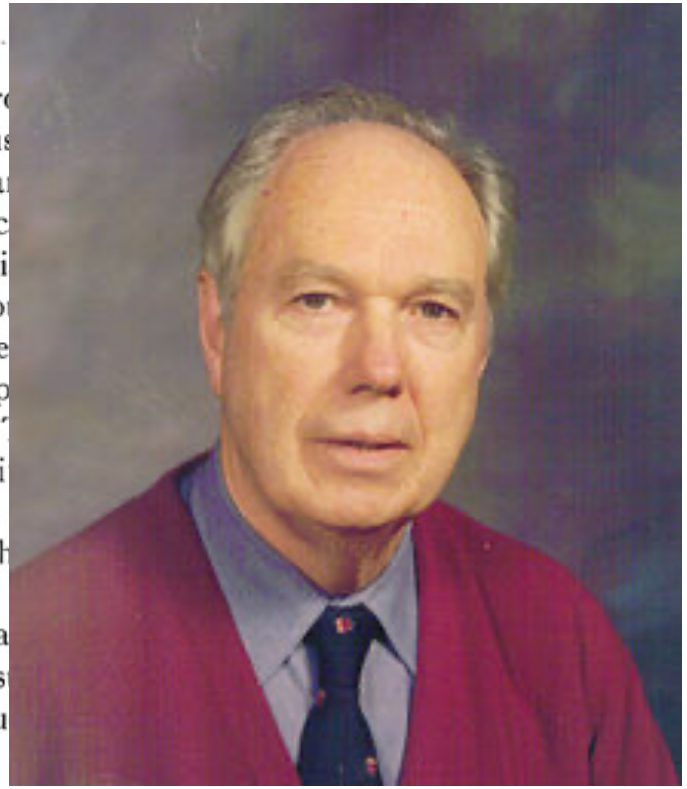
The parallel processors [single-instruction stream–multiple-data stream (SIMD)] are analyzed. In particular, a nesting type explanation is offered for Minsky's conjecture—the performance of a parallel processor increases as log M instead of M (the number of data stream processors).

Multiprocessors (MIMD) are subjected to a saturation syndrome based on general communications lockout. Simplified queuing models indicate that saturation develops when the fraction of task time spent locked out (L/E) approaches $1/n$, where n is the number of processors. Resources sharing in multiprocessors can be used to avoid

more "macr
particular us
must be sha
more signific

1) There i
limiting reso
est will eithe
tions will ap
figurations.
computer wi
cerned with
potential, wh
siderations.

2) We ma
sets. It is ass
set of instru

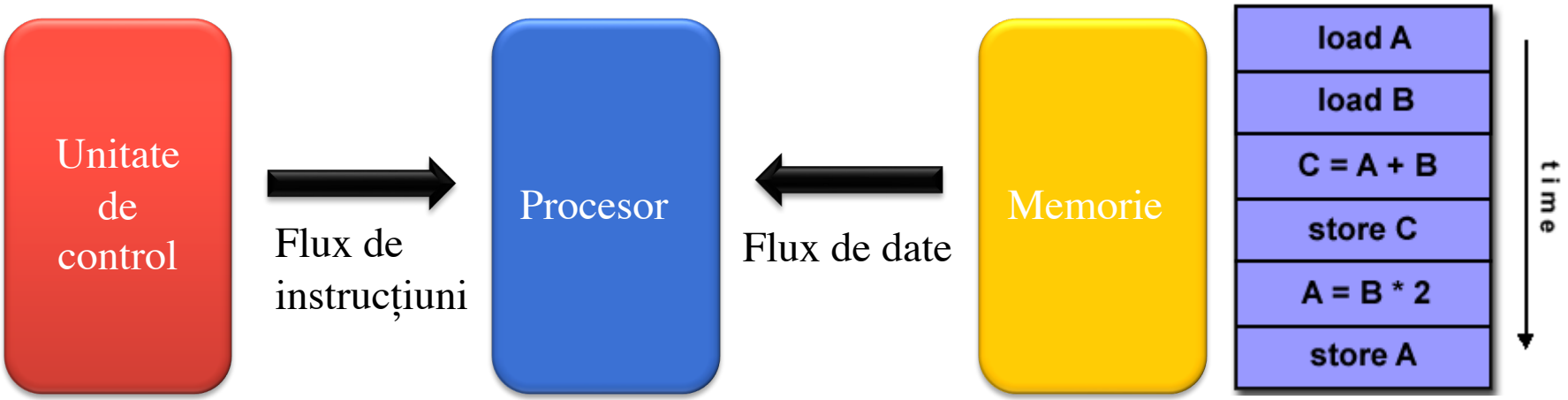


Taxonomia Flynn

- SISD
  - Single Instruction Stream, Single Data Stream
  - Calculatorul Clasic one-core
- SIMD
  - Single Instruction Stream, Multiple Data Streams
  - Suportul SSE; procesoare GPU
- MISD
  - Multiple Instruction Streams, Single Data Stream
  - Sisteme specializate
- MIMD
  - Multiple Instruction Streams, Multiple Data Streams
  - Procesoare actuale (ce aveți acasă și în buzunar)

SISD

- Model clasic Arhitectura *von Neumann*



SISD

- Model clasic Arhitectura *von Neumann*

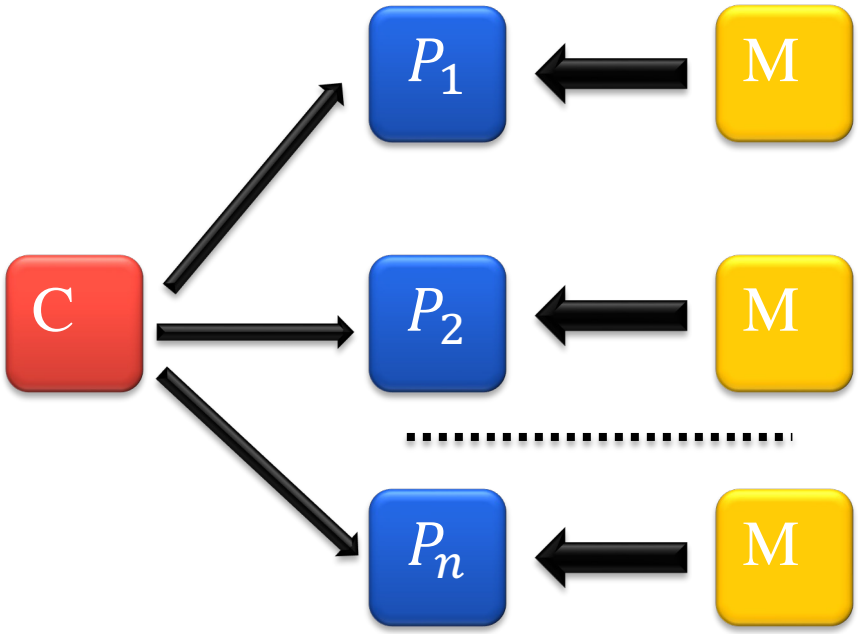
First Draft of a Report
on the EDVAC

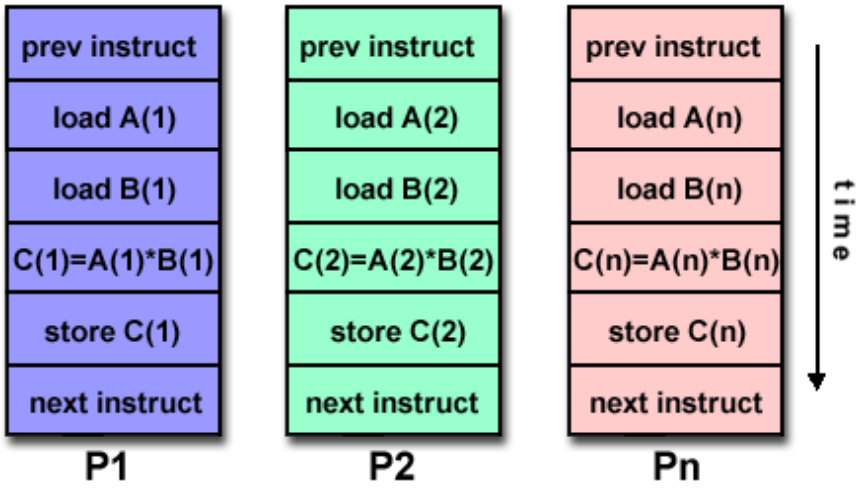
by

John von Neumann

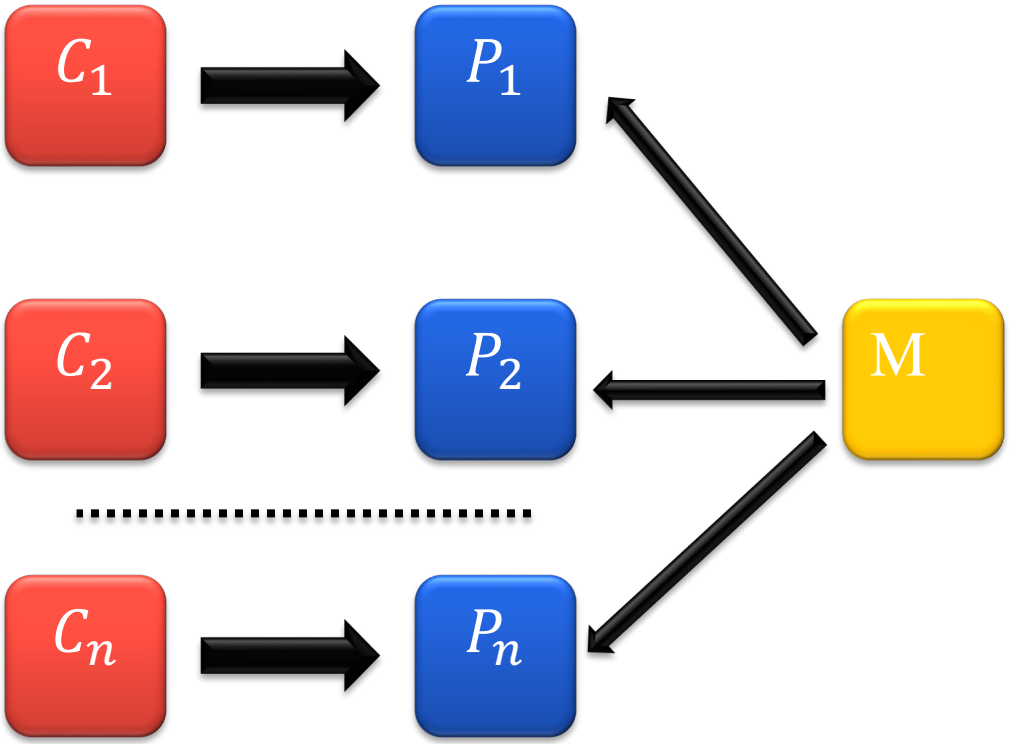


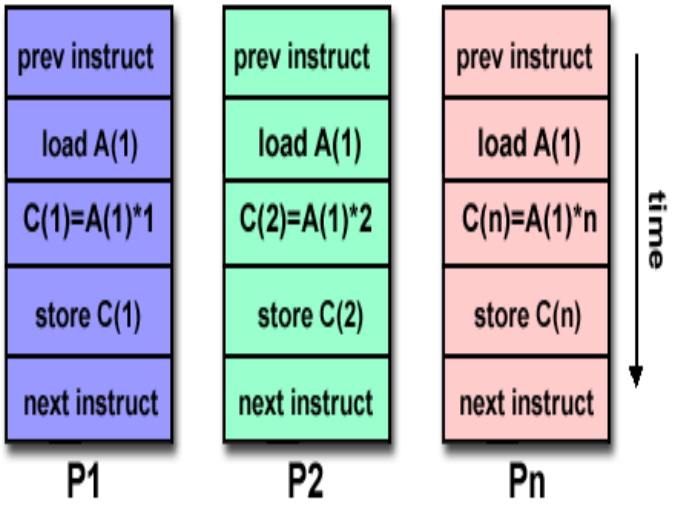
SIMD


C
P_1
P_2
P_n
M
M
M

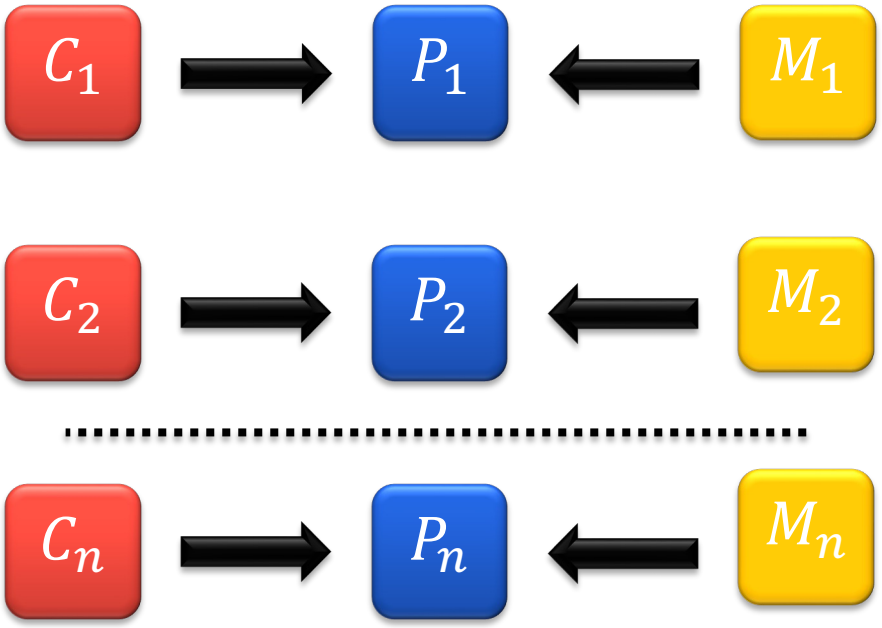

prev instruct
load A(1)
load B(1)
C(1)=A(1)*B(1)
store C(1)
next instruct
P1
prev instruct
load A(2)
load B(2)
C(2)=A(2)*B(2)
store C(2)
next instruct
P2
prev instruct
load A(n)
load B(n)
C(n)=A(n)*B(n)
store C(n)
next instruct
Pn
time

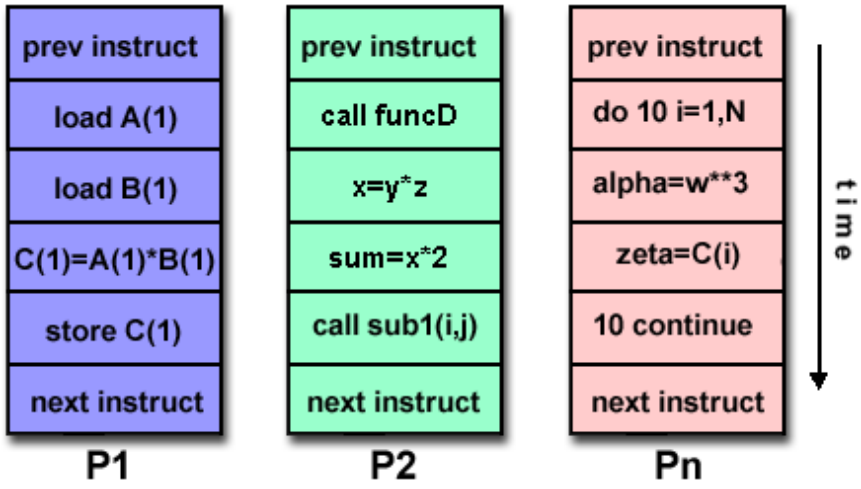
MISD


C_1
C_2
C_n
P_1
P_2
P_n
M


prev instruct
load A(1)
C(1)=A(1)*1
store C(1)
next instruct
P1
prev instruct
load A(1)
C(2)=A(1)*2
store C(2)
next instruct
P2
prev instruct
load A(1)
C(n)=A(1)*n
store C(n)
next instruct
Pn
time

MIMD


C_1
P_1
M_1
C_2
P_2
M_2
C_n
P_n
M_n

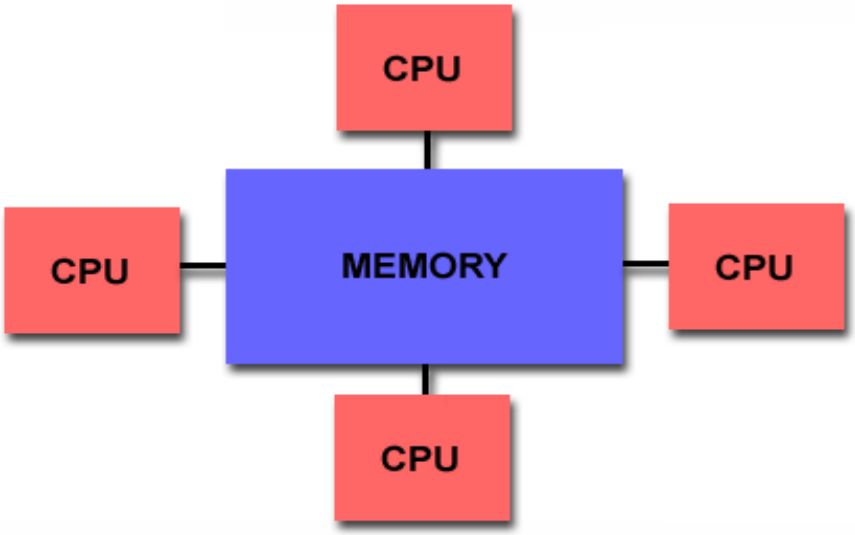

prev instruct
load A(1)
load B(1)
C(1)=A(1)*B(1)
store C(1)
next instruct
P1
prev instruct
call funcD
x=y*z
sum=x*2
call sub1(i,j)
next instruct
P2
prev instruct
do 10 i=1,N
alpha=w**3
zeta=C(i)
10 continue
next instruct
Pn
time

Memorie Partajată

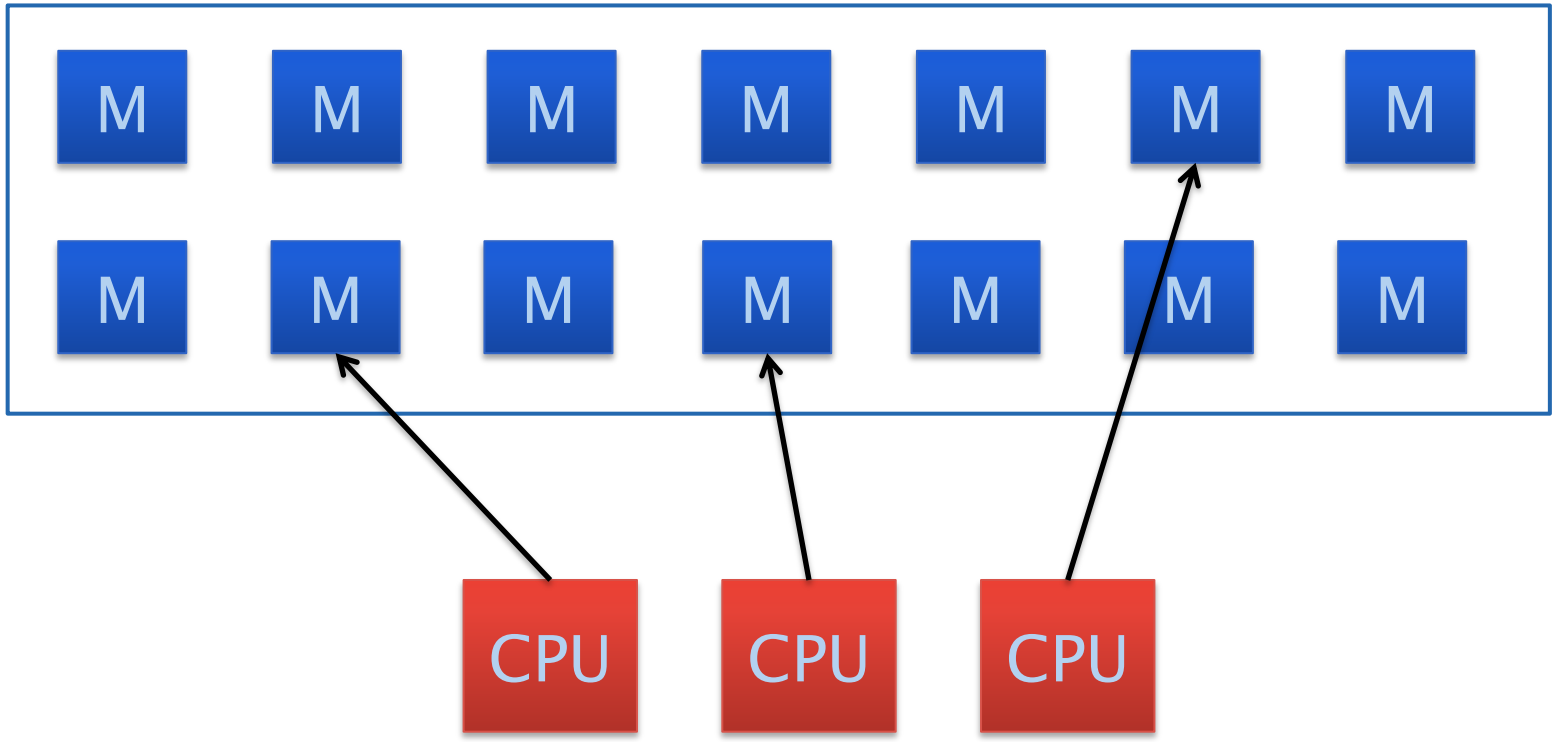
- Shared memory
  - Parallel Random Access Memory - **PRAM**
  - **EREW** – Exclusive Read Exclusive Write
  - **CREW** – **Concurrent Read Exclusive Write --** cel mai des întâlnit
  - **ERCW** – Exclusive Read Concurrent Write
  - **CRCW** – Concurrent Read Concurrent Write
  - O variabilă poate fi citită într-un pas în model **CR** dar în log(N) în model **ER**.

Memorie Partajată

- Uniform Memory Access
  - **UMA**

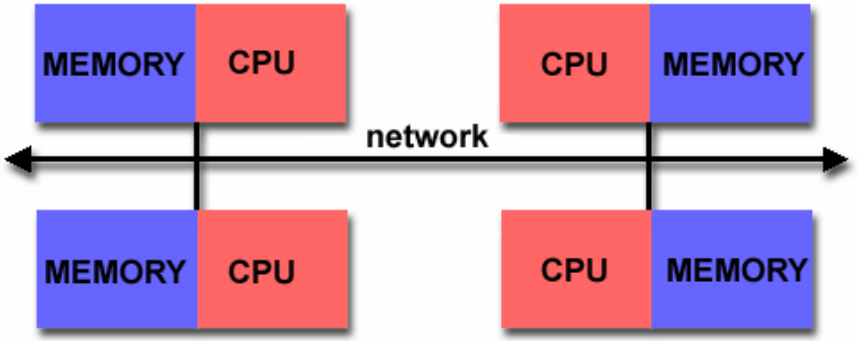


Memorie Partajată - Acces

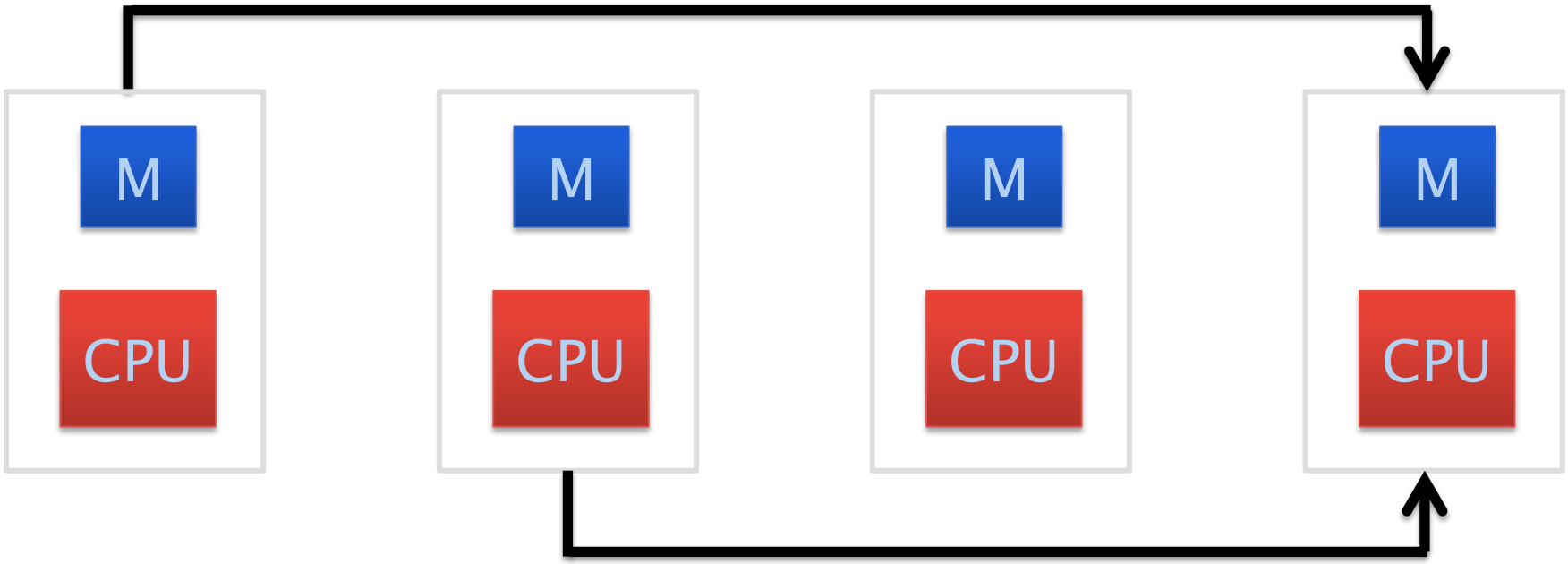


Memorie Distribuită

- Massively Parallel Processors
- Network of workstations
- Non-Uniform Memory Access
  - **NUMA**

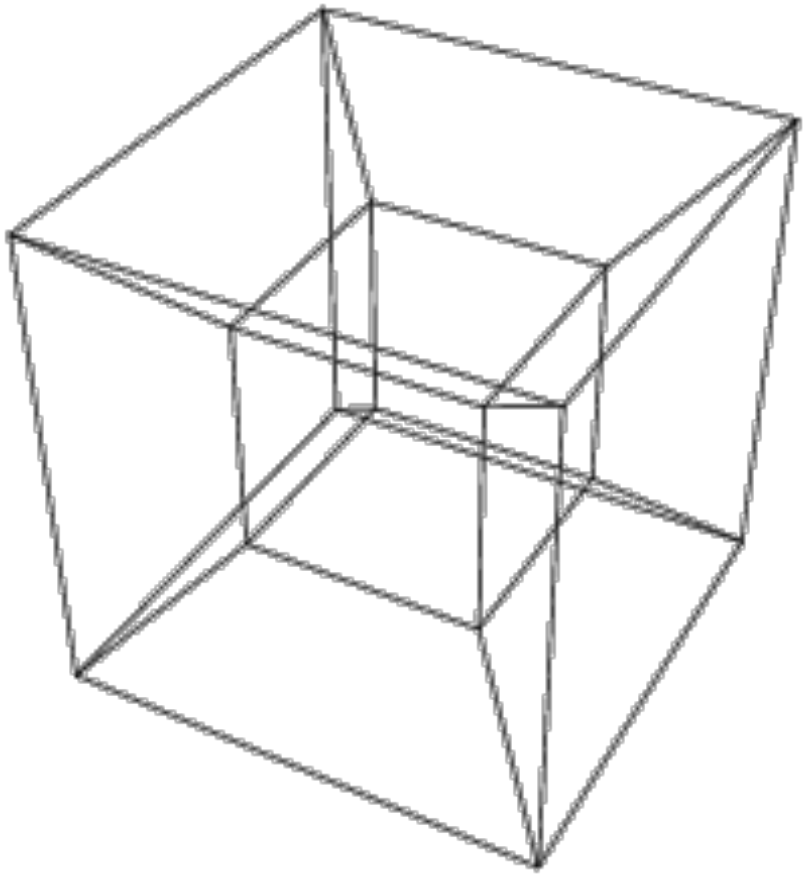


Memorie Distribuită - Acces



Rețele de configurare

- Topologii
  - Tablou
  - Arbore
  - Cub
  - Hipercub



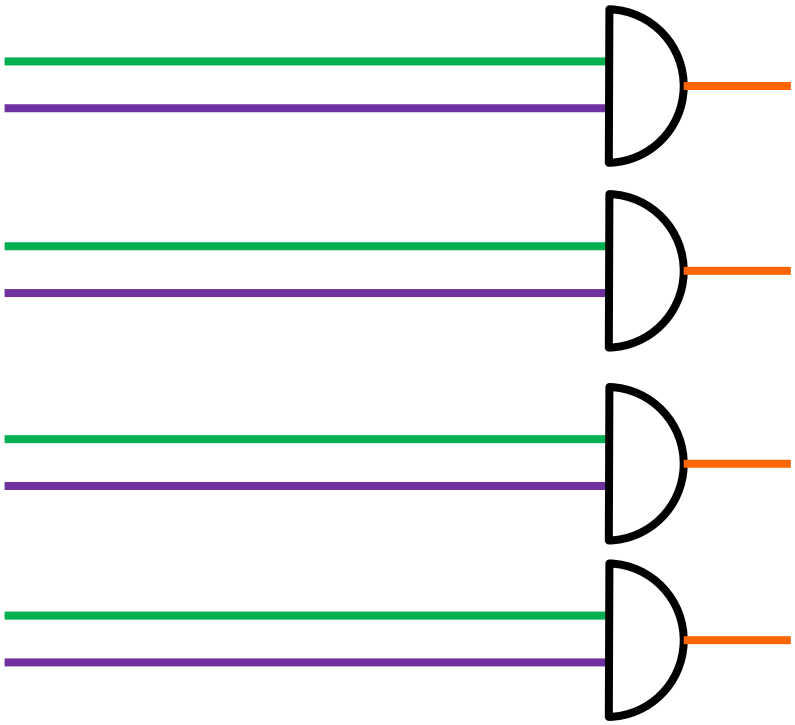
- Depinde de
  - Aplicație
  - Performanțe dorite
  - Număr procesoare disponibile
- Exemple: IBM 9000, Cray C90, Fujitsu VP

Tipuri de paralelism

- La nivel de bit
- La nivel de instrucțiune
- La nivel de task

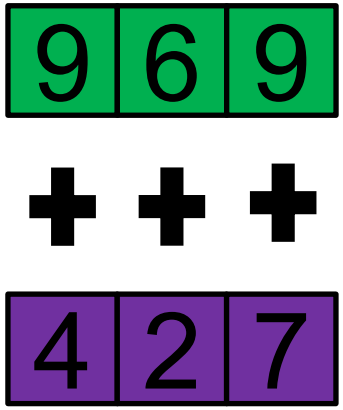
Tipuri de paralelism

- La nivel de bit
- La nivel de instrucțiune
- La nivel de task



Tipuri de paralelism

- La nivel de bit
- La nivel de instrucțiune
- La nivel de task



C = A + B

Adunarea a doi vectori

Tipuri de paralelism

- La nivel de bit
- La nivel de instrucțiune
- La nivel de task
  - **Multi-Tasking (pot comunica și procesele)**
  - **Multi-Threading**

Tipuri de paralelism

- La nivel de bit
- La nivel de instrucțiune
- La nivel de task
  - **Multi-Tasking (pot comunica și procesele)**
  - **Multi-Threading**

  Cum pot comunica două procese?

Notații pseudocod

```
co S1 || S2 || ... || Sn oc
```

Ex.1:

```
x=0; y=0;
co x=x+1 || y=y+1 oc
z=x+y;
```

Notații pseudocod

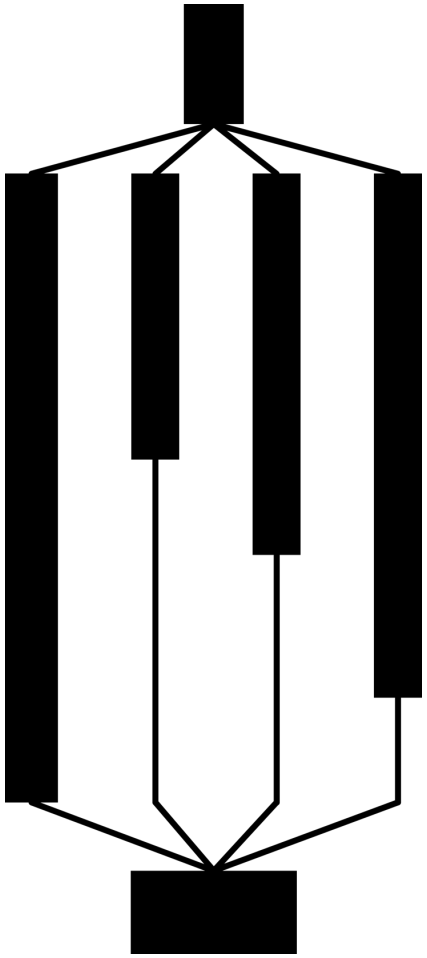
```
co [cuantificator]{Sj}
```

Ex. 2:

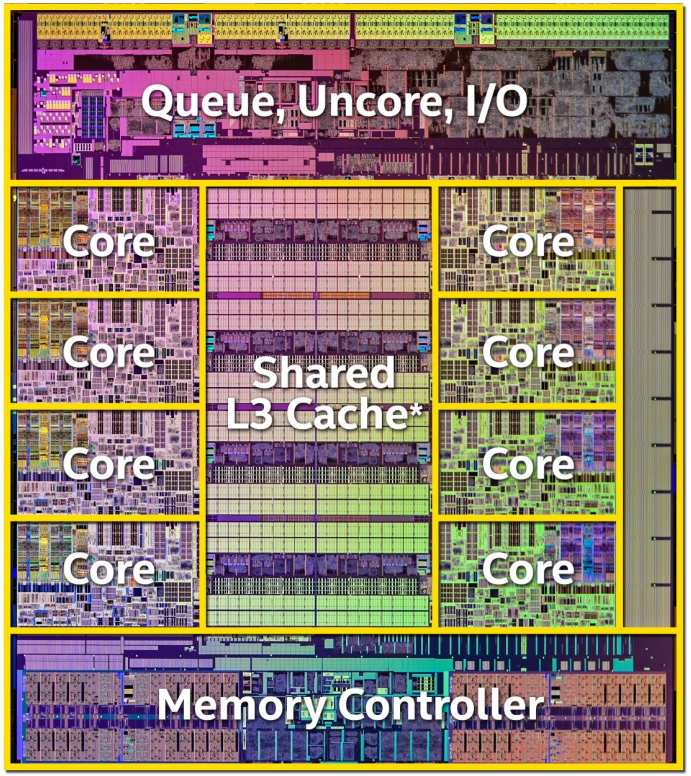
```
    co [j=1 to n] {a[j]=0; b[j]=0;}
```

```
process Name[cuantificatori]{ Sj }
```

Threads vs cores



New 8-Core Intel® Core™ i7 Processor Extreme Edition

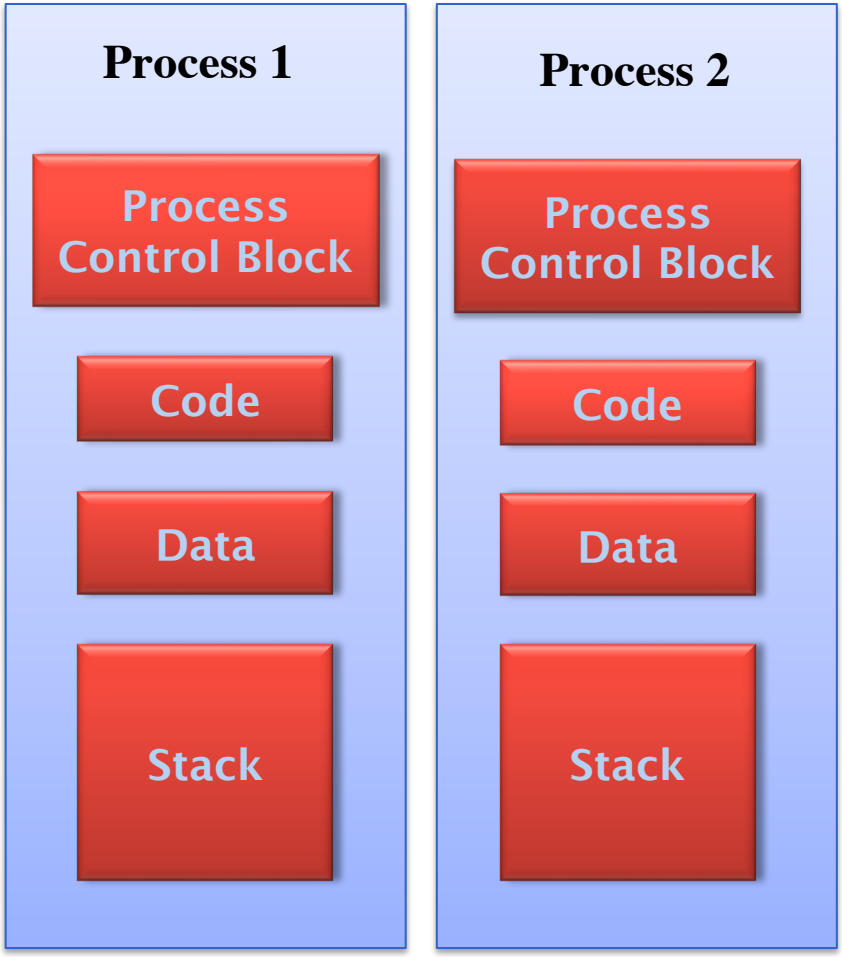


Intel® Core™ i7-5960X Processor Extreme Edition
Transistor count: 2.6 Billion
Die size: 17.6mm x 20.2mm

* 20MB of cache is shared across all 8 cores

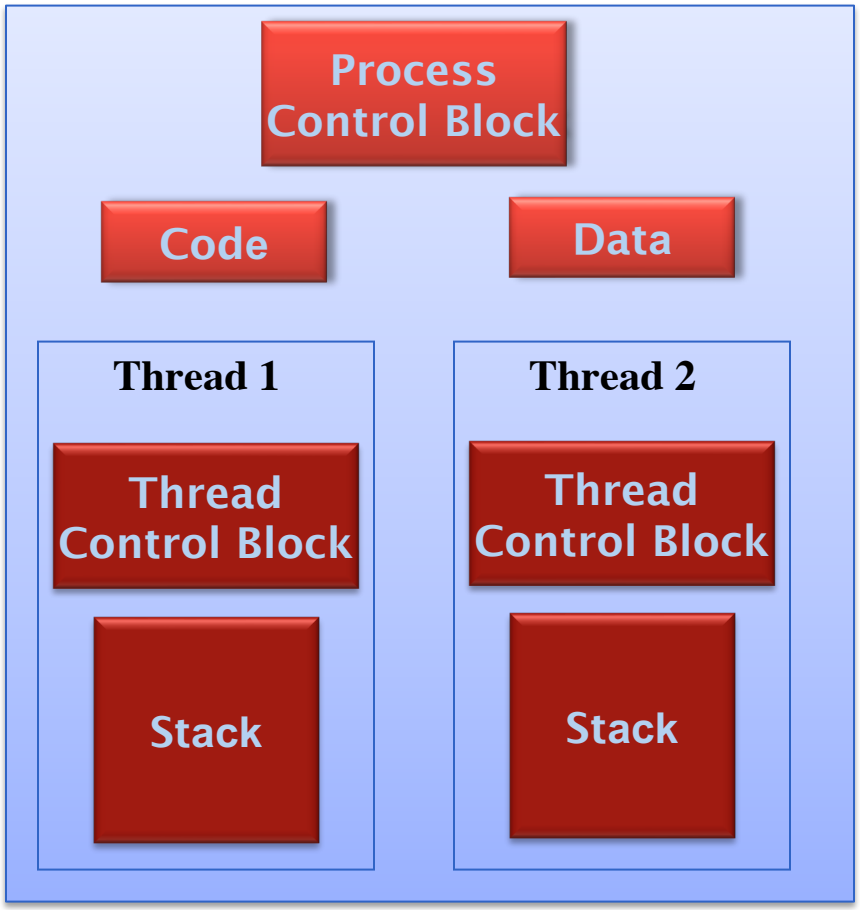
Procese și thread-uri

- Proces
  - Instanță a unui program în execuție
  - Pentru un proces, SO alocă:
    - Un spațiu în memorie (codul programului, zona de date, stiva)
    - Controlul anumitor resurse (fișiere, dispozitive I/O, …)

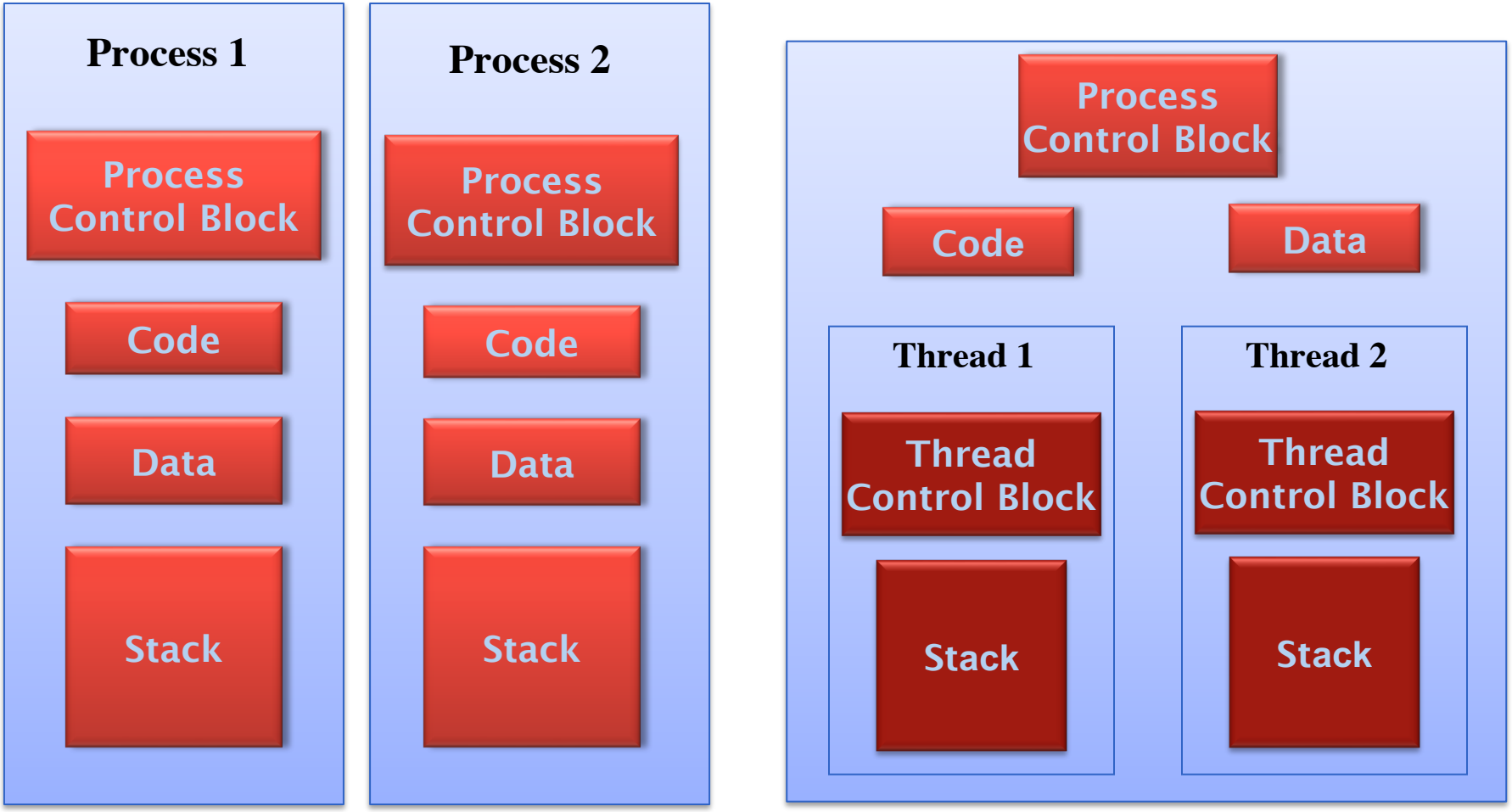


Thread (fir de execuție)

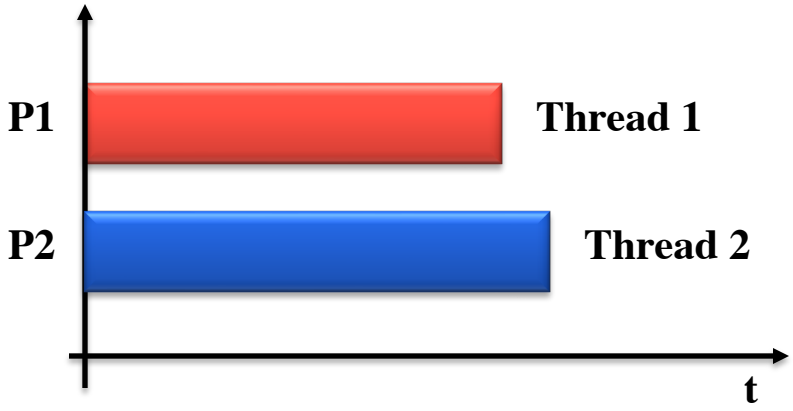
- Un proces poate avea mai multe fire de execuție
- Fir de execuție (thread):
  - Flux de control secvențial în cadrul unui proces
- Firele de execuție
  - împart același spațiu de adrese;
  - au în comun: zonele de cod și date din memorie, resursele procesului
  - zona de stivă – reprezintă starea curentă a thread-ului și este proprie thread-ului

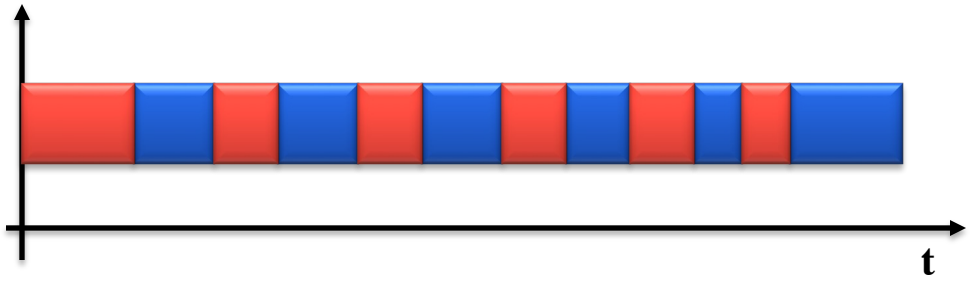


Multi-proces vs. multi-thread



Execuția thread-urilor





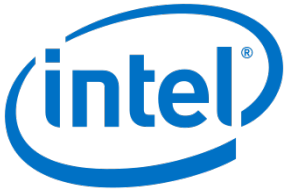
→ Putem considera execuție în paralel

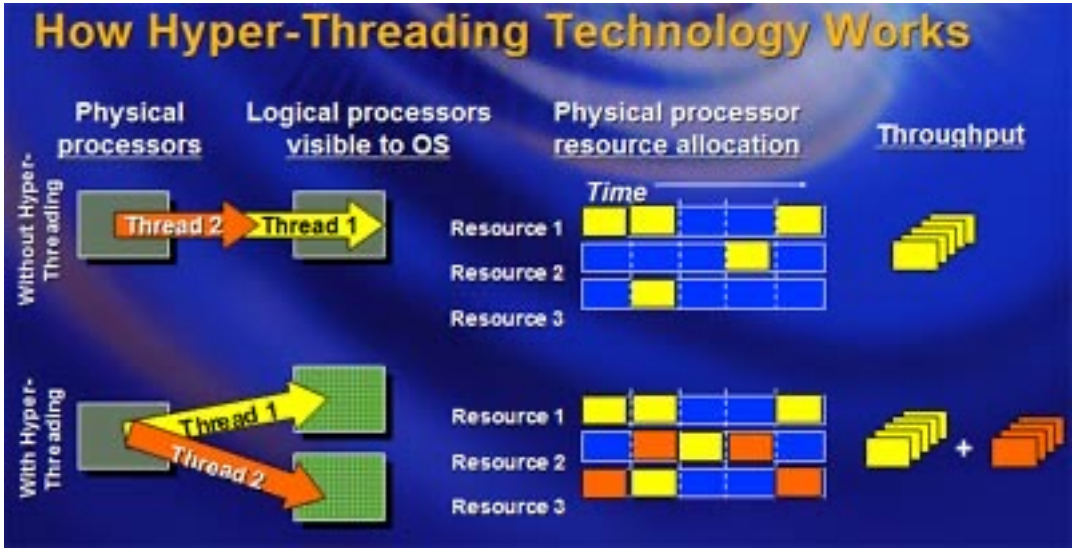
Defapt:

→ planificarea este realizată de către sistemul de operare

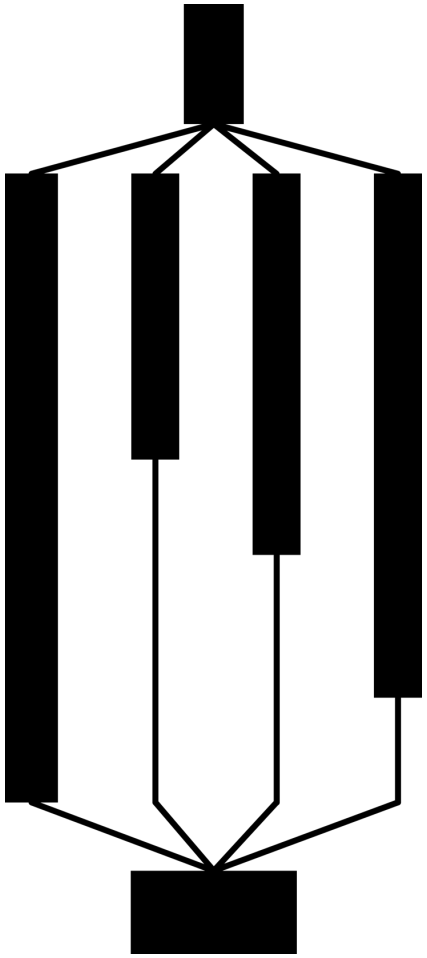
→ politici: divizarea timpului (*time sharing*), priorități

Hyperthreading – the confusion

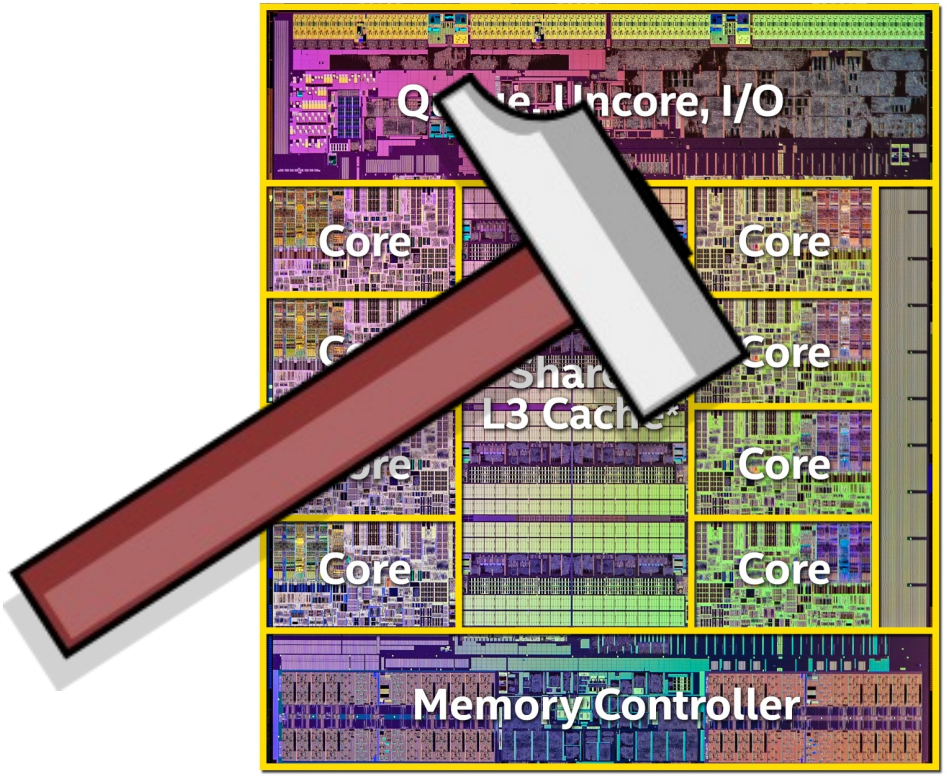
Thank you 



Hyperthreading – the confusion



New 8-Core Intel® Core™ i7 Processor Extreme Edition



Intel® Core™ i7-5960X Processor Extreme Edition
Transistor count: 2.6 Billion
Die size: 17.6mm x 20.2mm

* 20MB of cache is shared across all 8 cores

Multi-tasking vs Multi-threading





Multi-tasking

Task 1

Task 2



Multi-tasking

Task 2

Task 1



Multi-tasking

Task 1

Task 2



Multi-tasking

Task 2

Task 1



Multi-threading

Thread 2

Thread 3

Thread 4

Thread 1



Multi-threading

Thread 1

Thread 2

Thread 4

Thread 3



Multi-threading

Thread 1

Thread 2

Thread 4 Thread 3



Multi-threading

Thread 2

Thread 3

Thread 4

Thread 1



Multi-threading

Thread 1

Thread 4

Thread 2 Thread 3



Multi-tasking vs Multi-threading

Care sunt diferențele?

- Un proces are mai multe thread-uri
- Thread-urile unui proces împart memoria





Multi-tasking vs Multi-threading

Poți avea multi-tasking pe mai multe core-uri?

Poți avea mai multe thread-uri pe un singur core?





Multi-tasking vs Multi-threading

Poți avea multi-tasking pe mai multe core-uri? **DA**

Poți avea mai multe thread-uri pe un singur core? **DA**





Noțiuni cu care vom lucra

Race condition

Thread 1	Thread 2
a = a + 2	**a = a + 2**

Care este valoarea lui a în acest moment?

Race condition

a = 0

Thread 1	Thread 2
a = a + 2	**a = a + 2**

Care este valoarea lui a în acest moment?

Race condition

a = 0

Thread 1	Thread 2
a = a + 2	**a = a + 2**

What is the value of a?

4

Race condition

a = 0

Thread 1	Thread 2
a = a + 2	**a = a + 2**

What is the value of a?

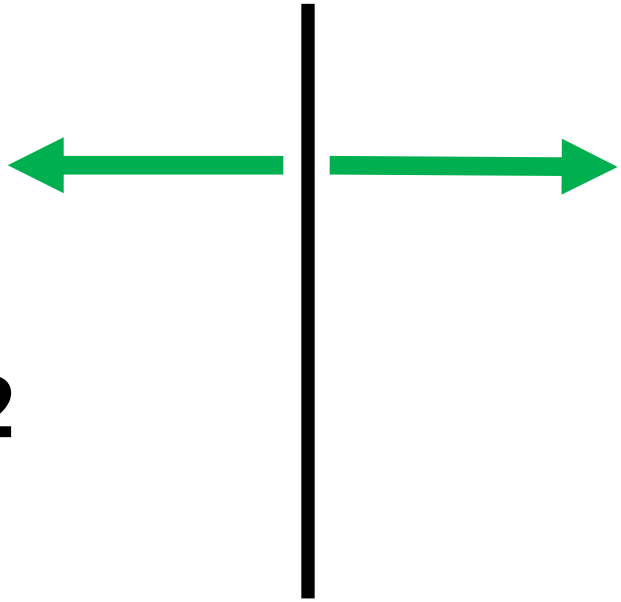
4 **AND** 2

Să privim mai bine

Race condition

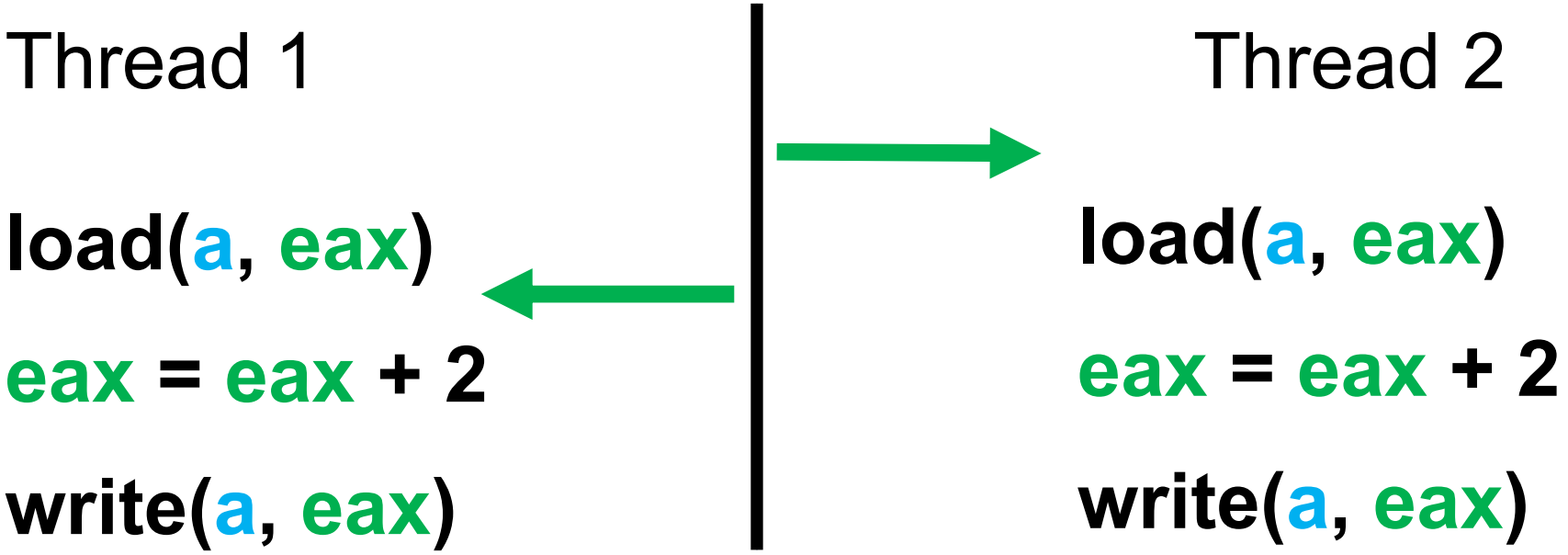
a = 0

Thread 1	Thread 2
load(a, eax)	**load(a, eax)**
eax = eax + 2	**eax = eax + 2**
write(a, eax)	**write(a, eax)**
eax =	**eax =**



Race condition

a = 0

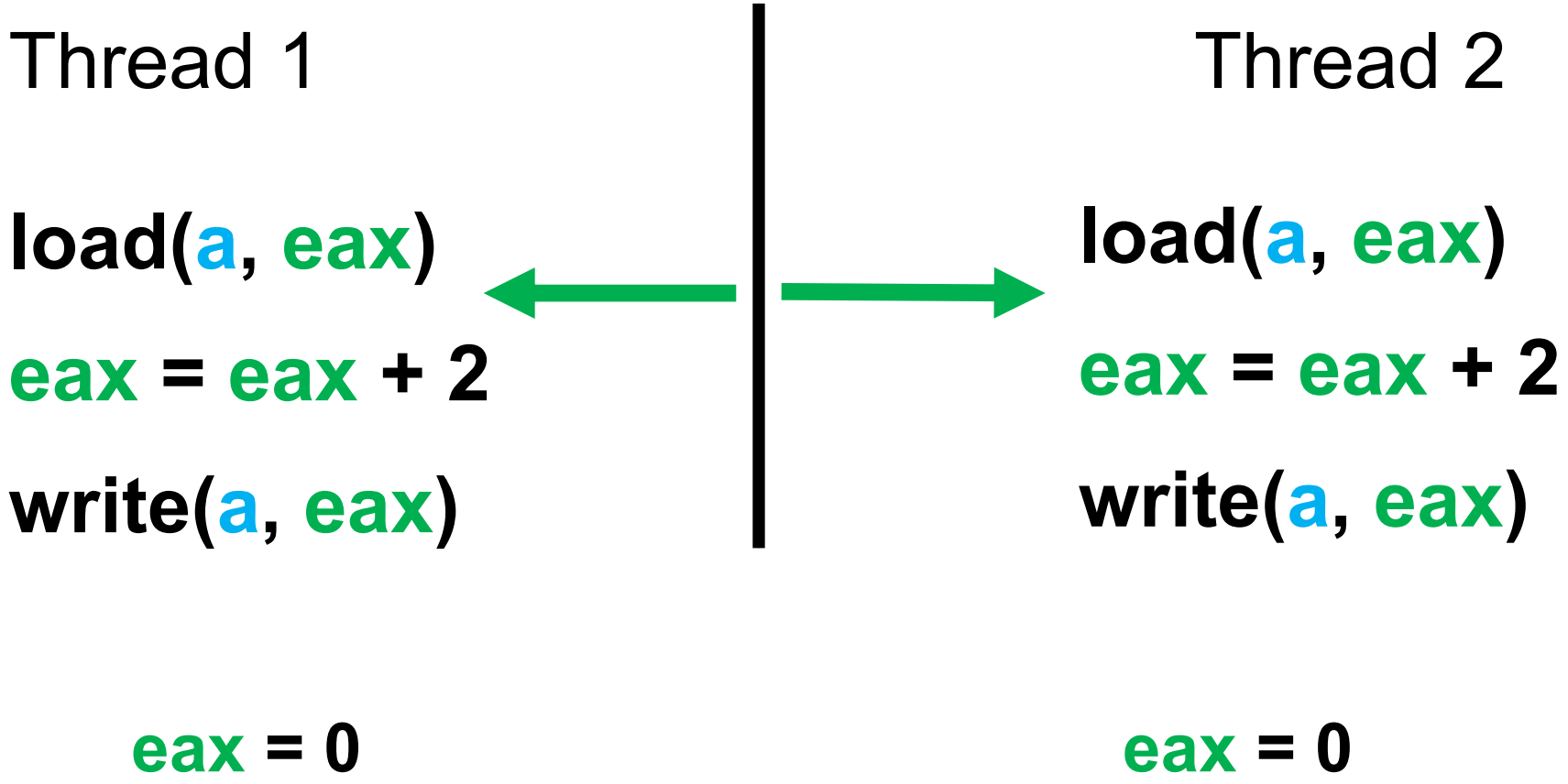


eax = 0

eax =

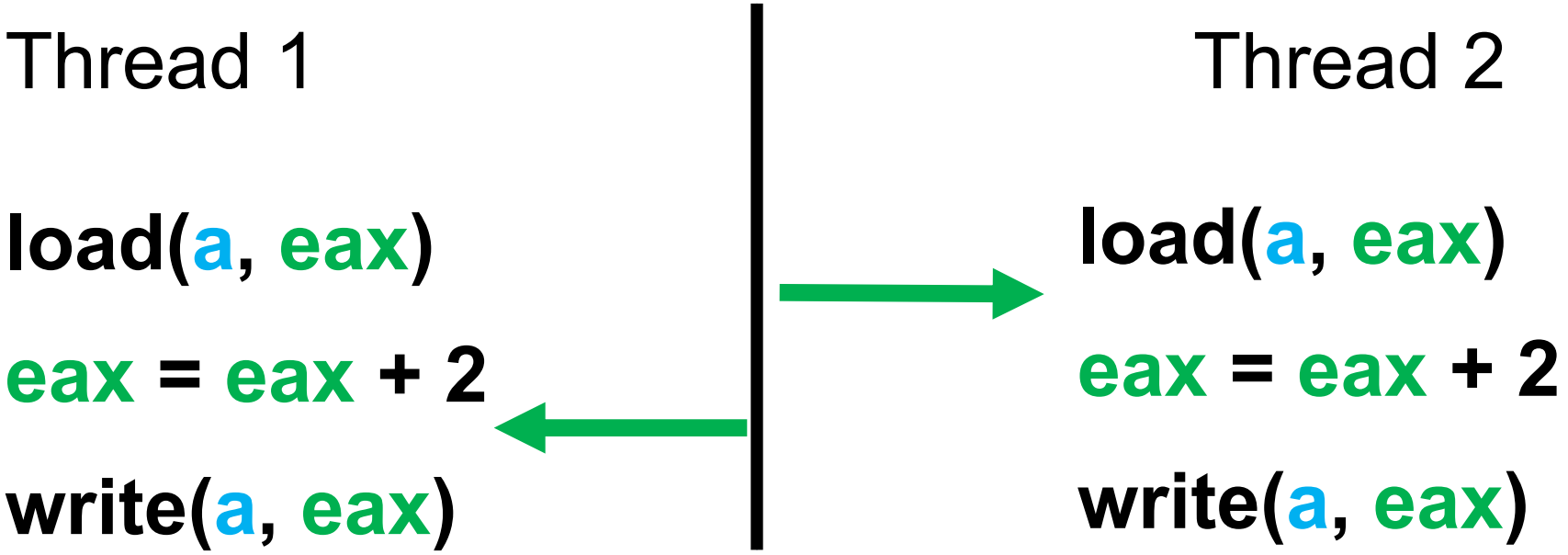
Race condition

a = 0



Race condition

a = 0

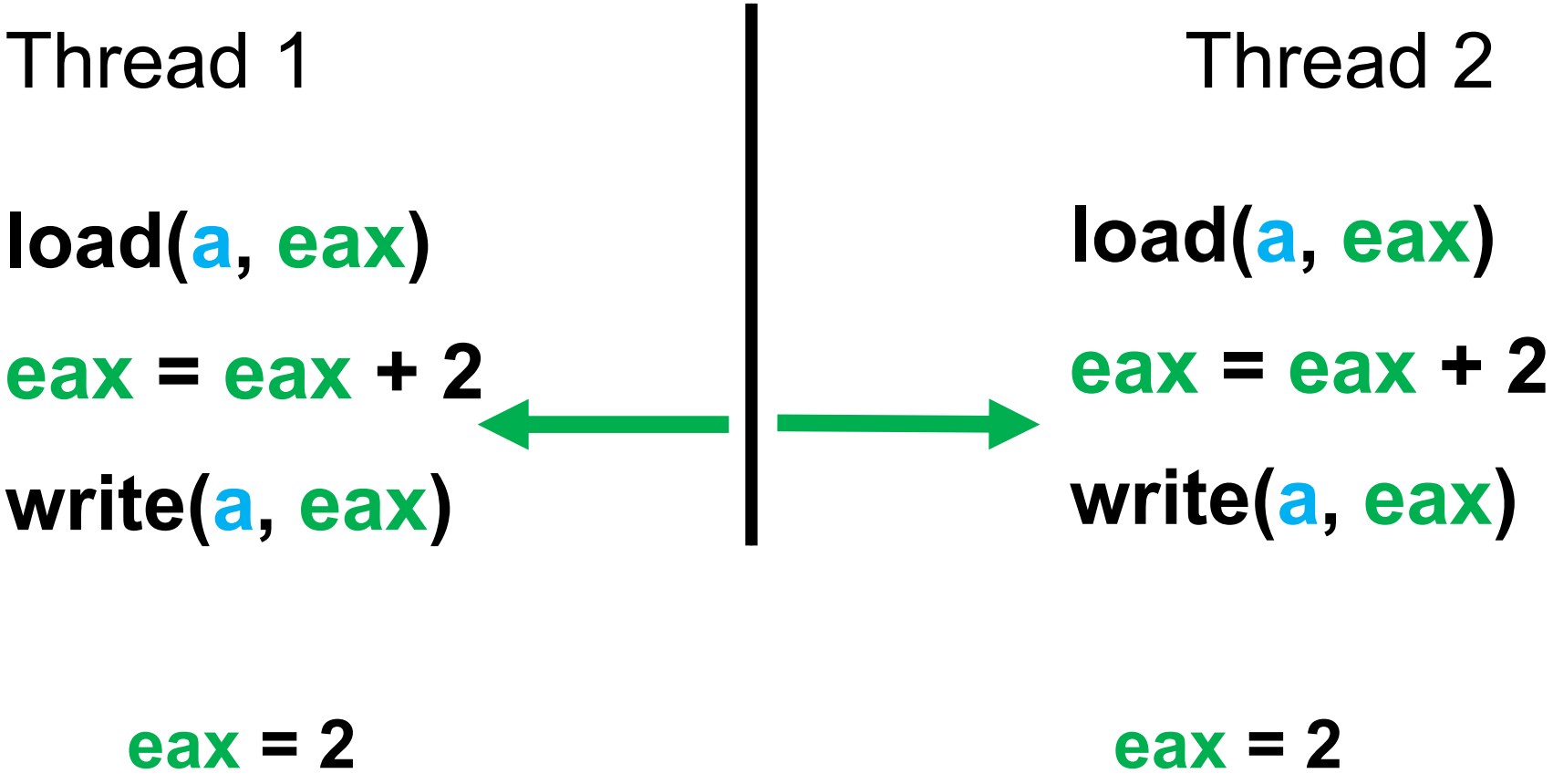


eax = 2

eax = 0

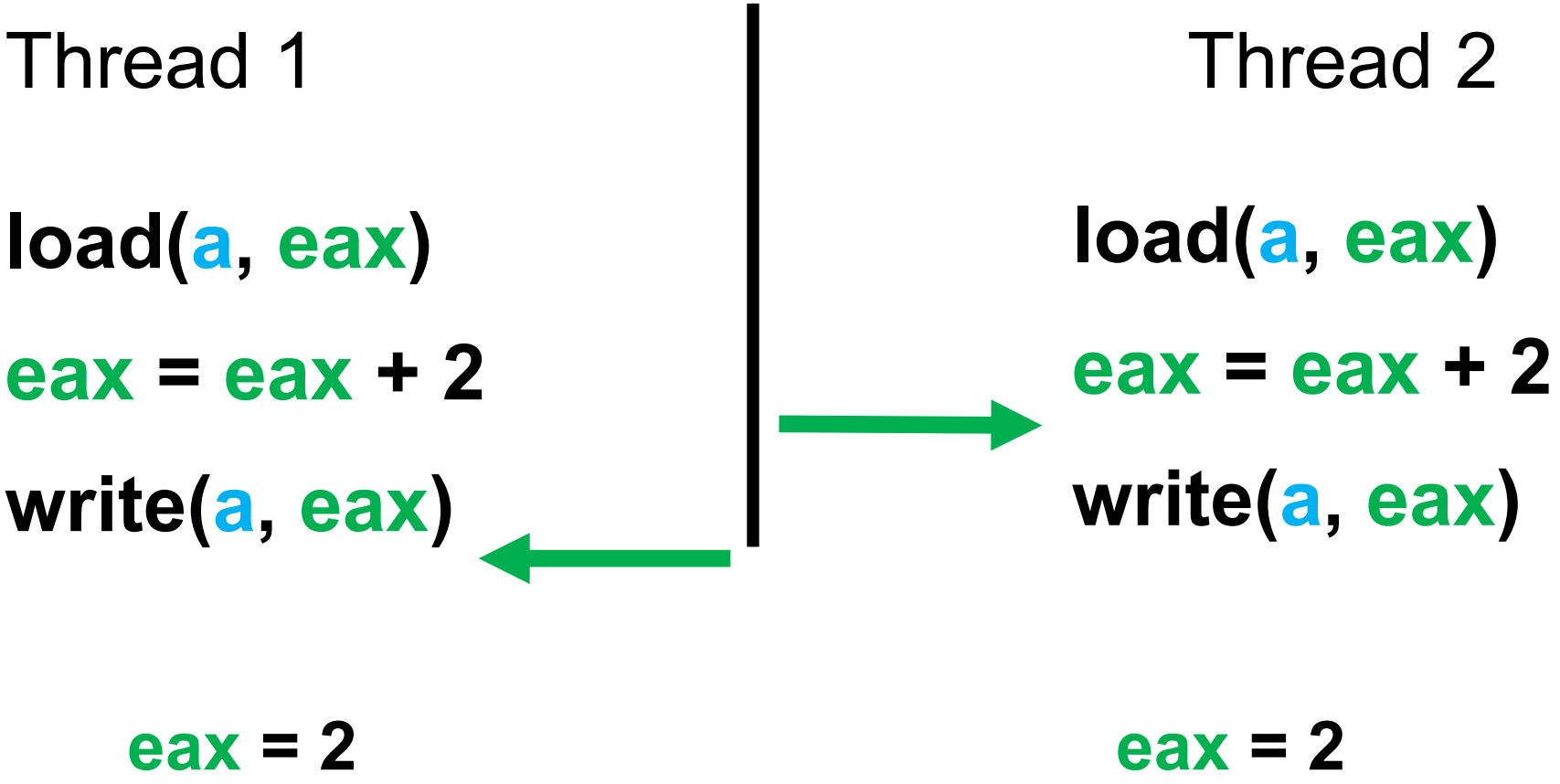
Race condition

a = 0



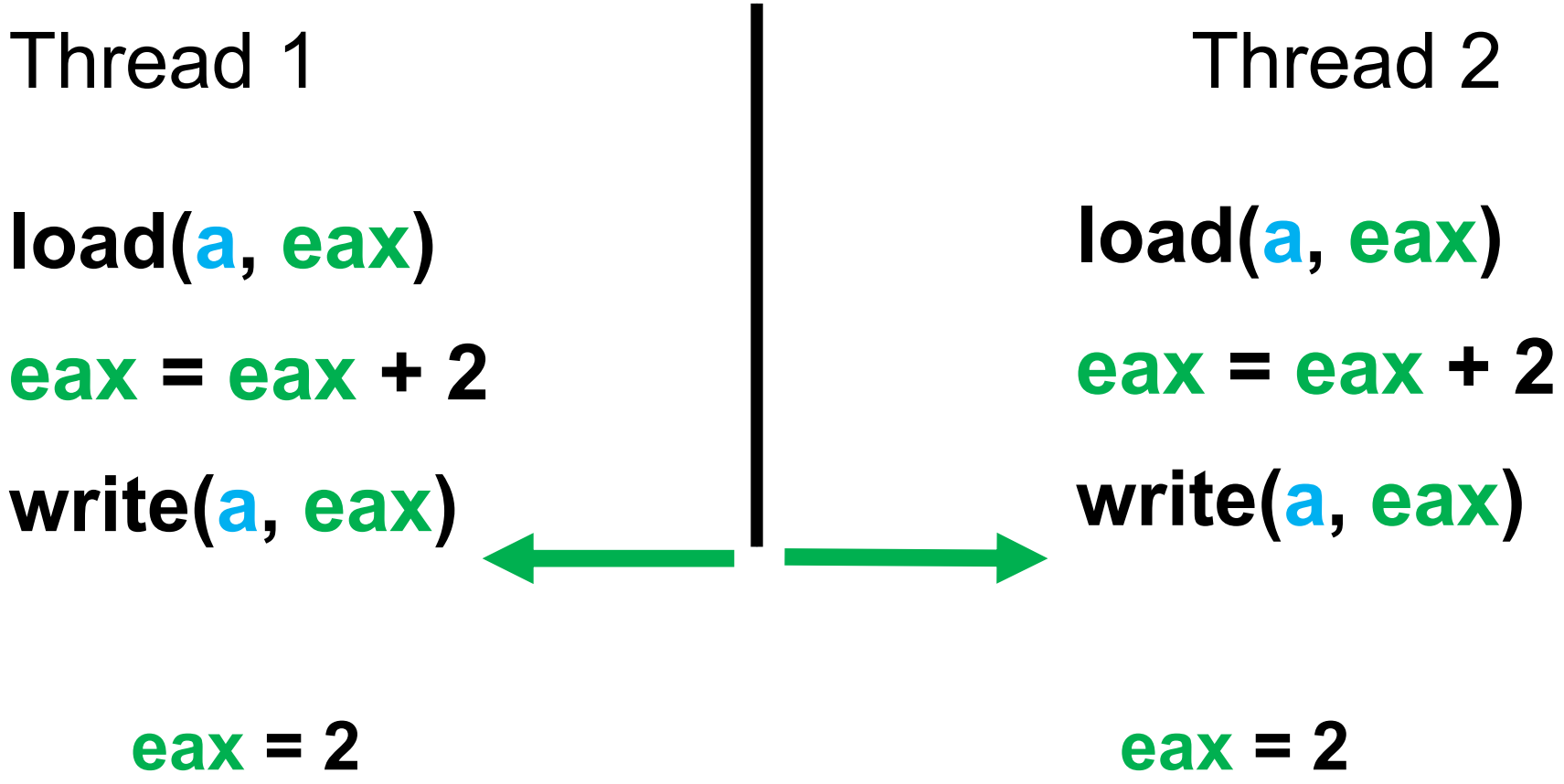
Race condition

a = 2



Race condition

a = 2

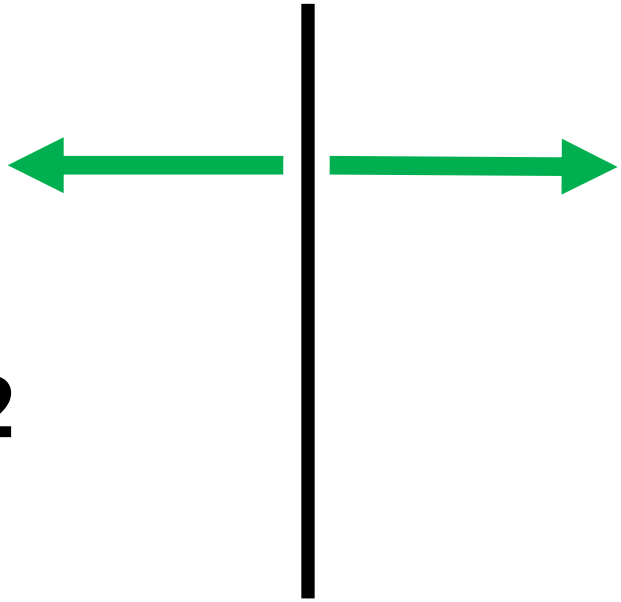


Să vedem o altă variantă de rulare

Race condition

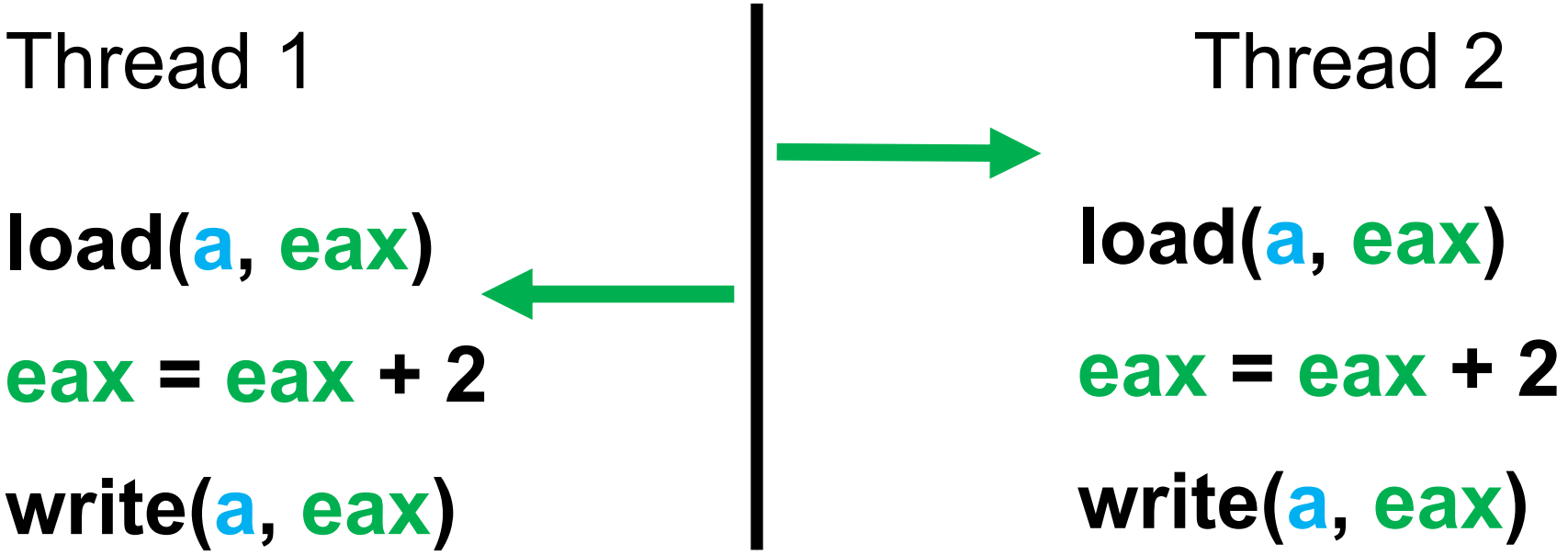
a = 0

Thread 1	Thread 2
load(a, eax)	**load(a, eax)**
eax = eax + 2	**eax = eax + 2**
write(a, eax)	**write(a, eax)**
eax =	**eax =**



Race condition

a = 0

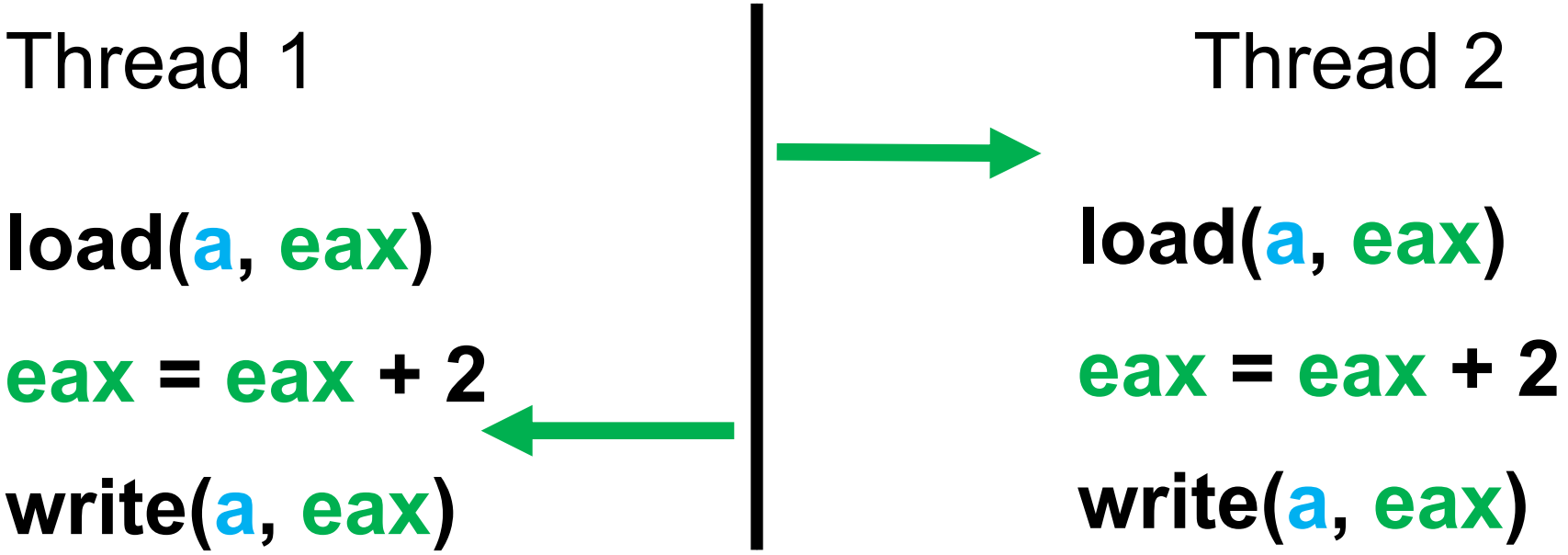


eax = 0

eax =

Race condition

a = 0

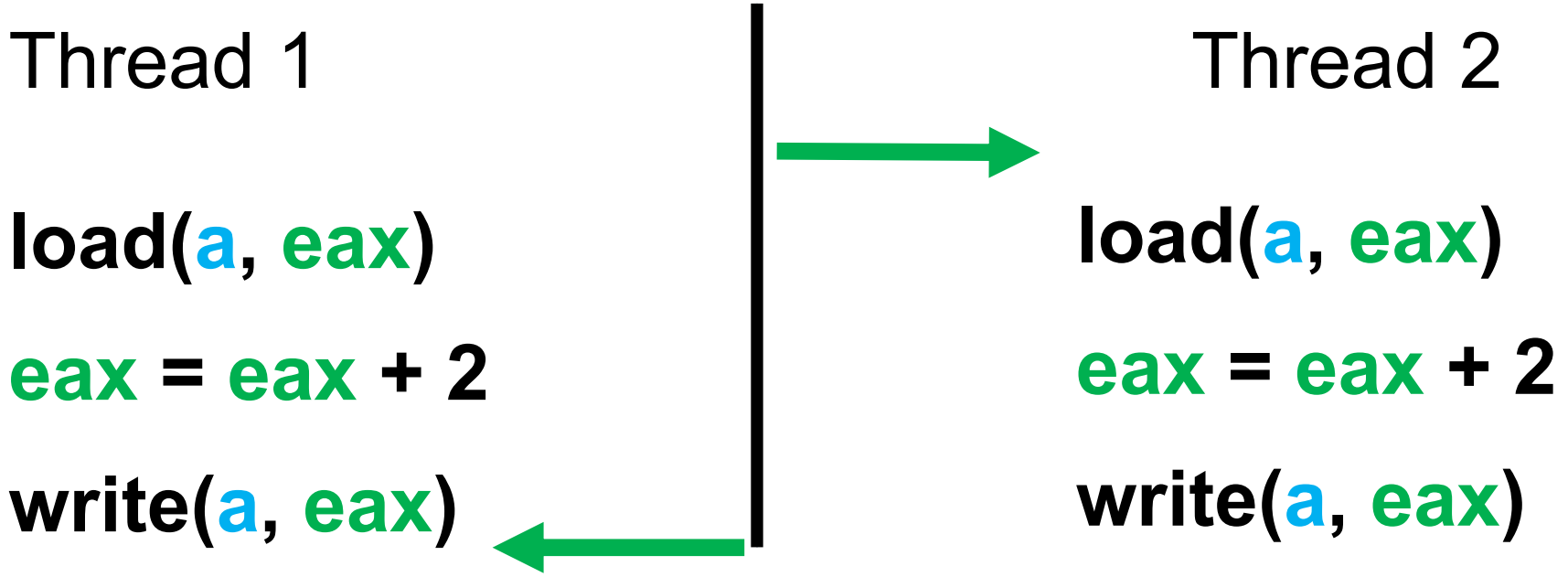


eax = 2

eax =

Race condition

a = 2

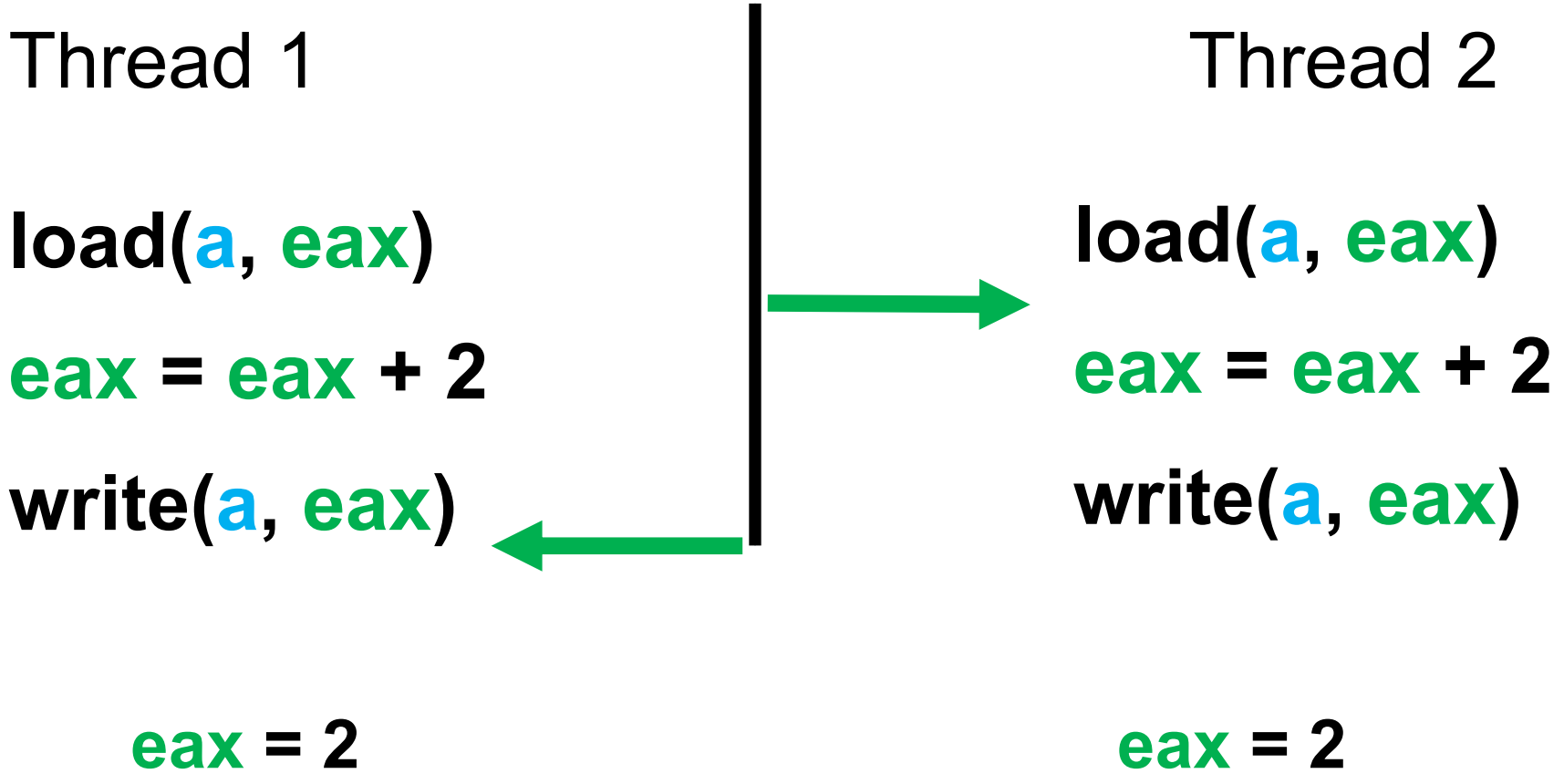


eax = 2

eax =

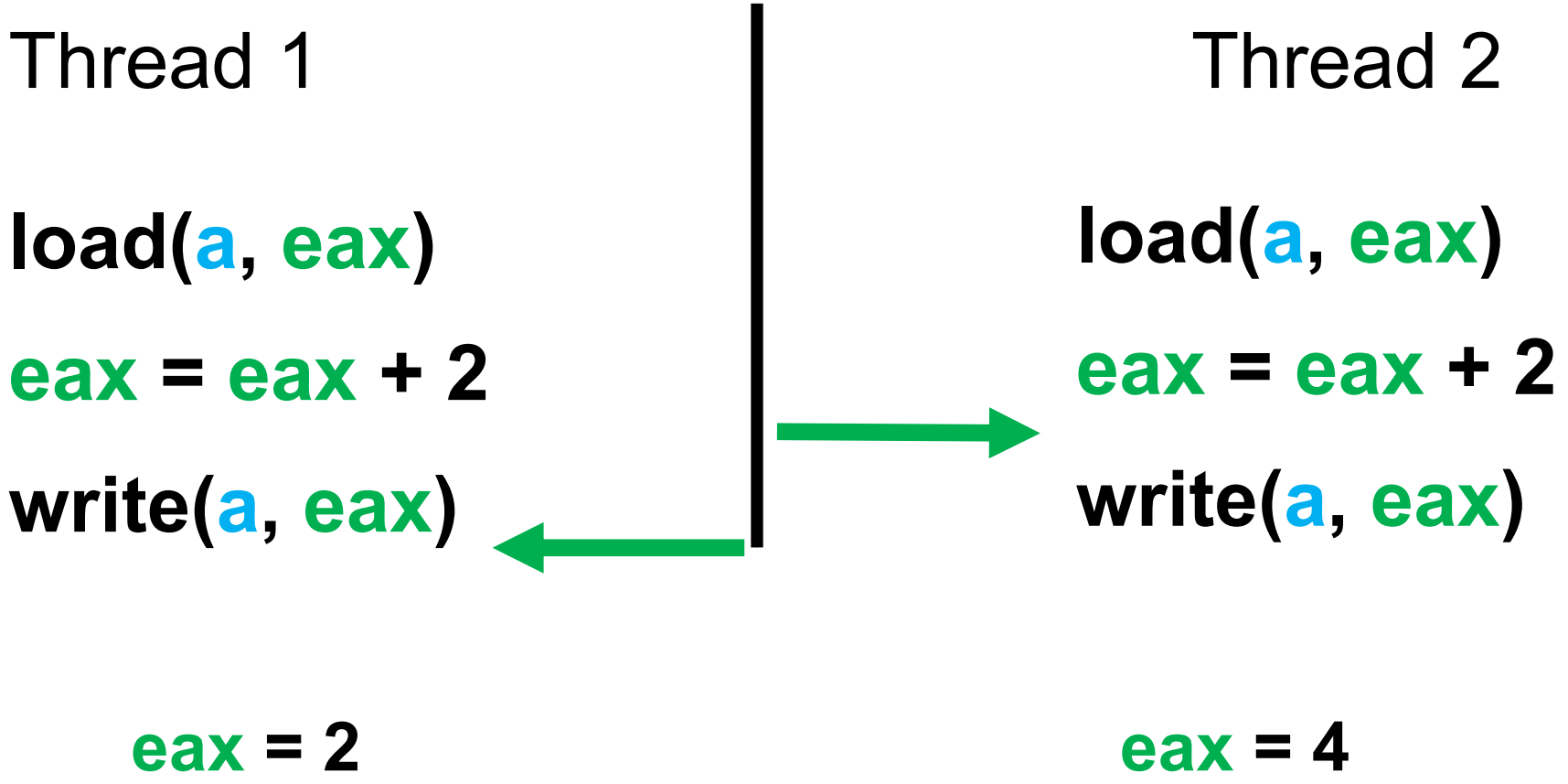
Race condition

a = 2



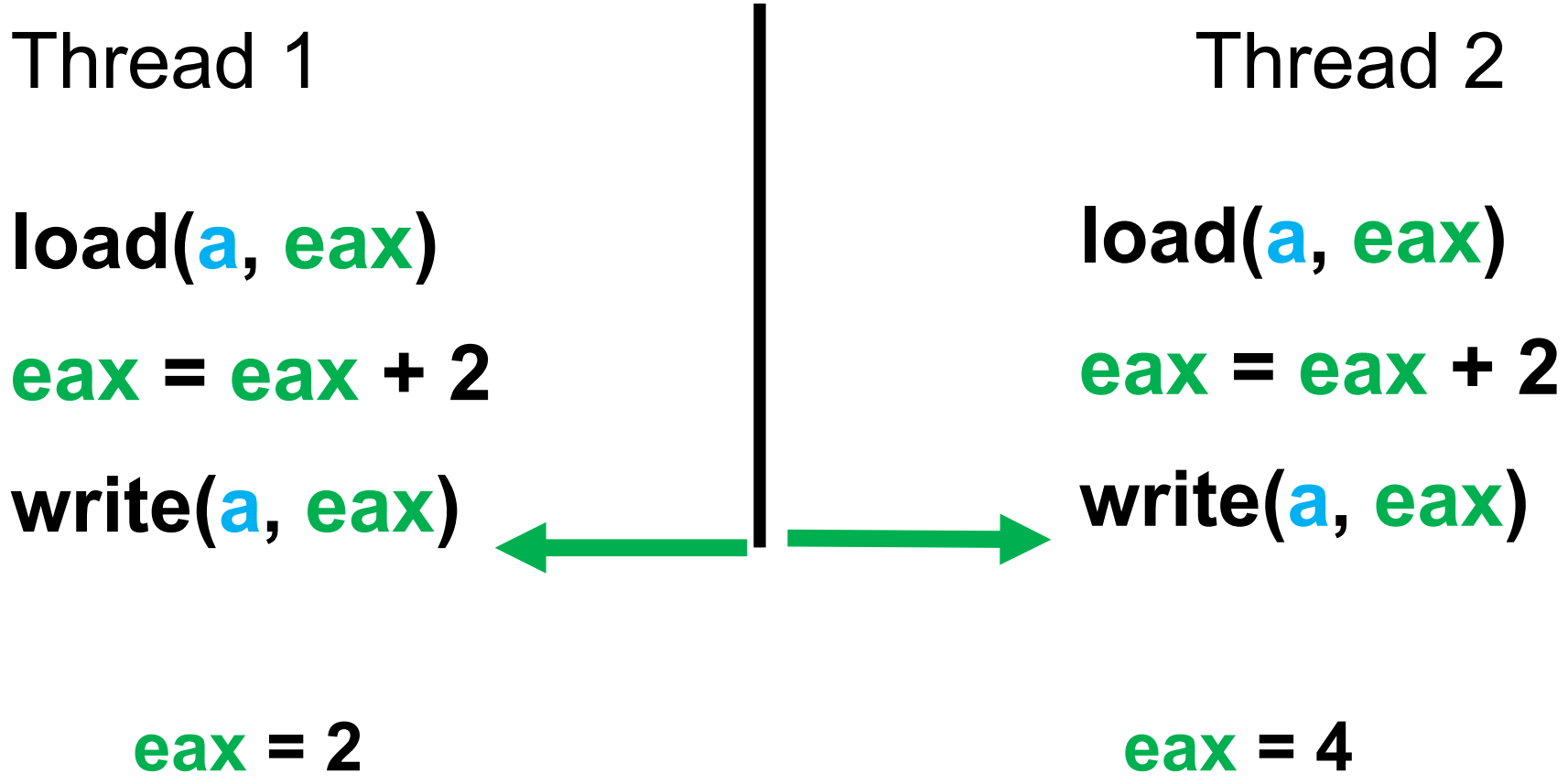
Race condition

a = 2



Race condition

a = 4



Primitive de sincronizare

Synchronization primitives

- Atomics
- Semaphore 
  - Binary semaphore (Mutex)
  - Critical section
- Barrier 

Atomics

- Fie operația de 64 biți pe un procesor de 64 biți **C = A + B**

```
load(A, eax)
load(B, ebx)
eax = eax + ebx
write(C, eax)
```

Atomics

- Fie operația de 64 biți pe un procesor de 32 biți **C = A + B**

```
load(A[0], eax)
load(B[0], ebx)
eax = eax + ebx
write(C[0], eax)
load(A[1], eax)
load(B[1], ebx)
eax = eax + ebx
write(C[1], eax)
```

Atomics

- Fie operația de 64 biți pe un procesor 32 biți **C = A + B**

```
load(A[0], eax)
load(B[0], ebx)
eax = eax + ebx
write(C[0], eax)
load(A[1], eax)
load(B[1], ebx)
eax = eax + ebx
write(C[1], eax)
```



Putem avea doar jumătate de **C** modificat

Atomics

- Fie operația de 64 biți pe un procesor 32 biți **C = A + B**

```
load(A[0], eax)
load(B[0], ebx)
eax = eax + ebx
write(C[0], eax)   <-
load(A[1], eax)
load(B[1], ebx)
eax = eax + ebx
write(C[1], eax)   <-
```

Atomicitatea asigură că **C** va fi vizibil doar complet modificat, sau complet nemodificat.

Excludere mutuală - mutex

Mutual exclusion – soluția lui Dekker

EWD35 - 1

EWD35.htm

About the sequentiality of process descriptions.

Over de sequentialiteit van procesbeschrijvingen.

Het is niet ongebruikelijk, wanneer een spreker zijn
een inleiding. Omdat ik mij hier mogelijk richt tot een g
minder vertrouwd is met de problematiek, die ik wil aansn
die ik zal moeten gebruiken, wilde ik in dit geval ter in
inleidingen houden, nl. een om de achtergrond van de prob
een tweede, om U een gevoel te geven voor de aard van de
wij tegen het lijf zullen lopen.



Theodorus (Dirk) J. Dekker

Soluția lui Dekker

```
wants_to_enter[0] = true
while (wants_to_enter[1]) {
        if (turn != 0) {
                wants_to_enter[0] = false
                while (turn != 0) { // busy wait }
                wants_to_enter[0] = true
        }
}
// critical section ...
turn = 1
wants_to_enter[0] = false
```

Mutual exclusion – soluția lui Dijsktra

Solution of a Problem in Concurrent Programming Control

E. W. Dijkstra
Technological University, Eindhoven, The Netherlands

A number of mainly independent sequential-cyclic processes with restricted means of communication with each other can be made in such a way that at any moment one and only one of them is engaged in the "critical section" of its cycle.

Introduction

Given in this paper is a solution to a problem for which, to the knowledge of the author, has been an open question since at least 1962, irrespective of the solvability. The paper consists of three parts: the problem, the solution, and the proof. Although the setting of the problem might seem somewhat academic at first, the author trusts that anyone familiar with the logical problems that arise in

computer can only request one one-way message at a time. And only this w
this problem is f

The Solution

The common
"Boo
The integer
will only be set
by the others. It
well outside the
mentioned set to
The program

```
"integer j;
Li0:  b[i] := fals
Li1:  if k ≠ i the
Li2:  begin c[i] :
Li3:  if b[k] then
      go to Li1
      end
        else
```



Soluția lui Dijsktra

b[i] = true, c[i] = true, sw[i] = true

```
b[i] = false
while (sw[i]) {
      sw[i] = false
      if (k != i) {
            c[i] = true
            if (b[k])
                  k = i
            sw[i] = true
      } else {
            c[i] = false
            for (j = 0; j < N; j++)
                  if (j != i && !c[j])
                        sw[i] = true
   }
}
//critical
b[i] = true
c[i] = true
```

lock() - P

unlock() - V

Soluția lui Peterson

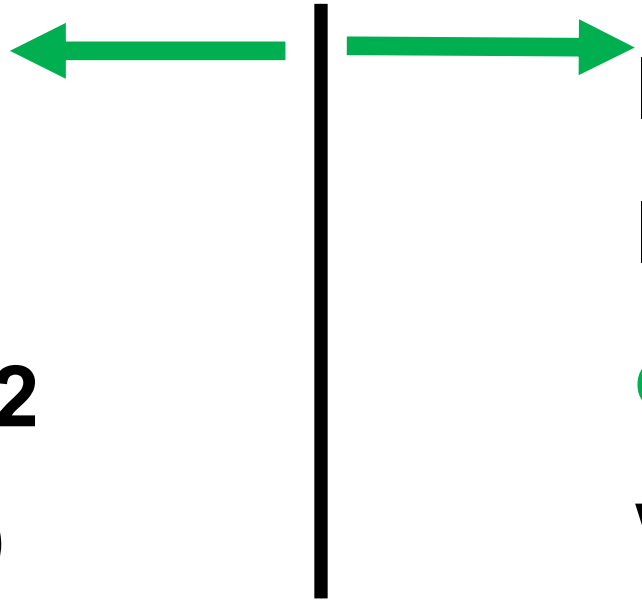
```
flag[0] = true;
P0_gate: turn = 1;
while (flag[1] && turn == 1) { // busy wait }
// critical section ...
flag[0] = false;
```

Soluție cu asistare hardware

```
while (test_and_set(lock));
// critical section
lock = 0
```

Race condition – soluție

a = 0



Thread 1	Thread 2
lock(locka)	lock(locka)
load(a, eax)	load(a, eax)
eax = eax + 2	eax = eax + 2
write(a, eax)	write(a, eax)
unlock(locka)	unlock(locka)
eax = 0	eax = 0

Race condition – soluție



a = 0

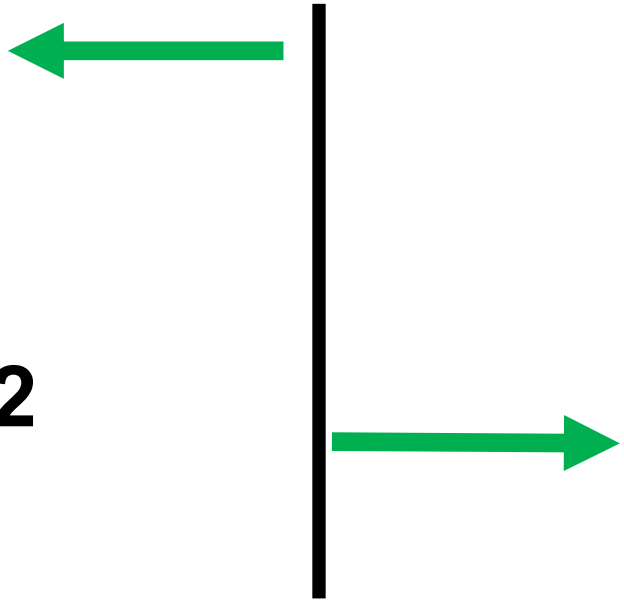
Thread 1	Thread 2
lock(locka)	lock(locka)
load(a, eax)	load(a, eax)
eax = eax + 2	eax = eax + 2
write(a, eax)	write(a, eax)
unlock(locka)	unlock(locka)
eax = 0	eax = 2

Race condition – soluție

OK

a = 0

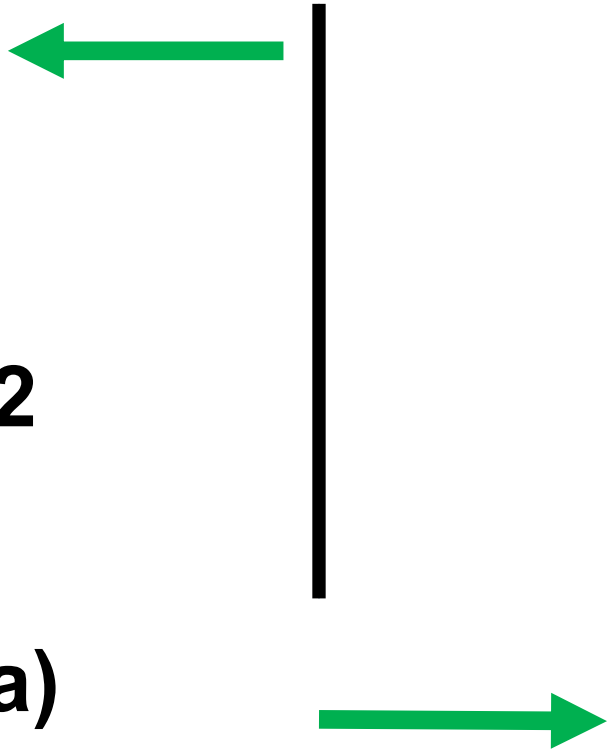
Thread 1	Thread 2
lock(locka)	lock(locka)
load(a, eax)	load(a, eax)
eax = eax + 2	eax = eax + 2
write(a, eax)	write(a, eax)
unlock(locka)	unlock(locka)
eax = 0	eax = 2



Race condition – soluție

a = 2

Thread 1	Thread 2
lock(locka)	**lock(locka)**
load(a, eax)	**load(a, eax)**
eax = eax + 2	**eax = eax + 2**
write(a, eax)	**write(a, eax)**
unlock(locka)	**unlock(locka)**
eax = 0	**eax = 2**



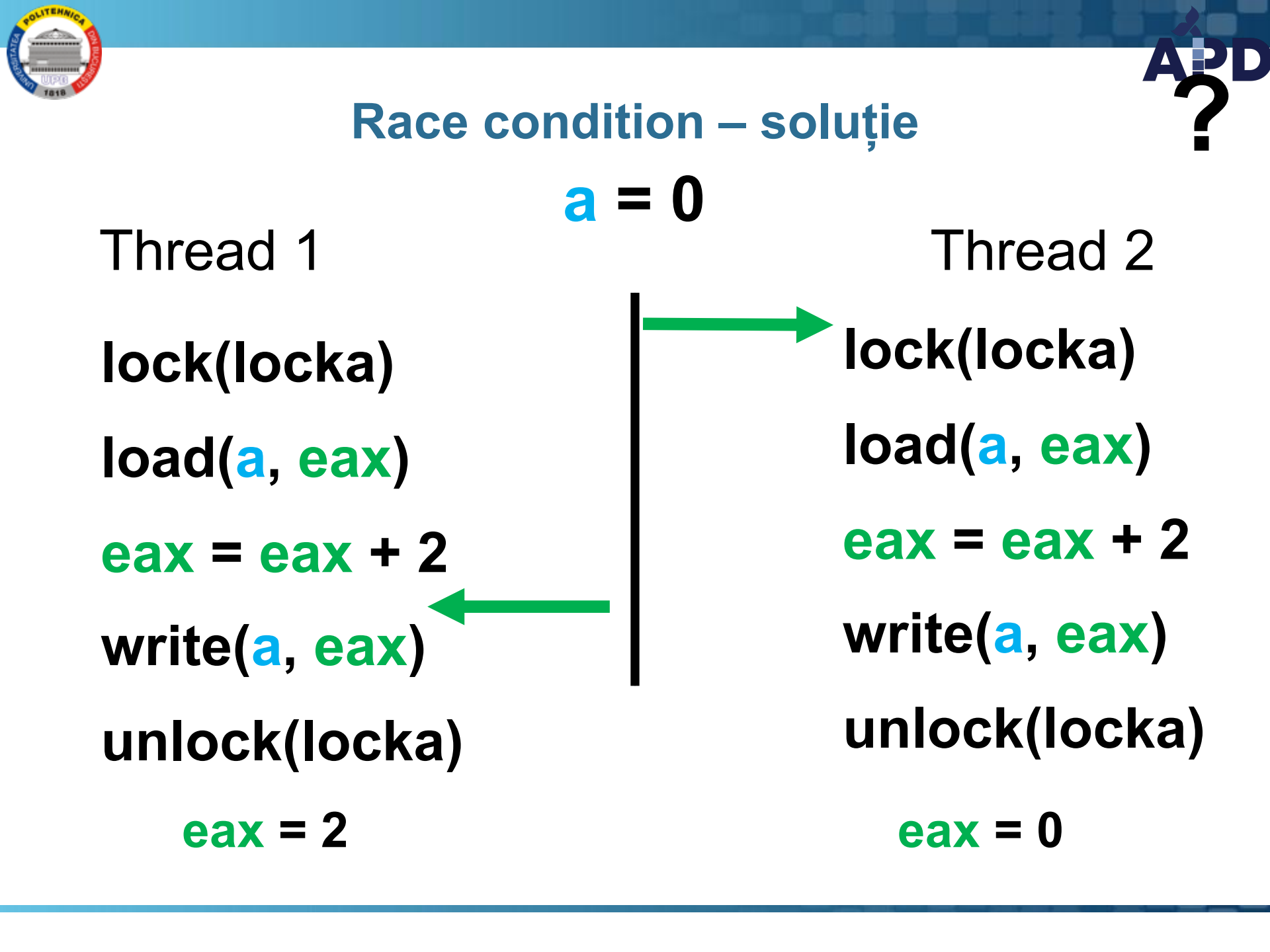
Race condition – soluție

OK

a = 2

Thread 1	Thread 2
lock(locka)	lock(locka)
load(a, eax)	load(a, eax)
eax = eax + 2	eax = eax + 2
write(a, eax)	write(a, eax)
unlock(locka)	unlock(locka)
eax = 0	eax = 2

Race condition – soluție



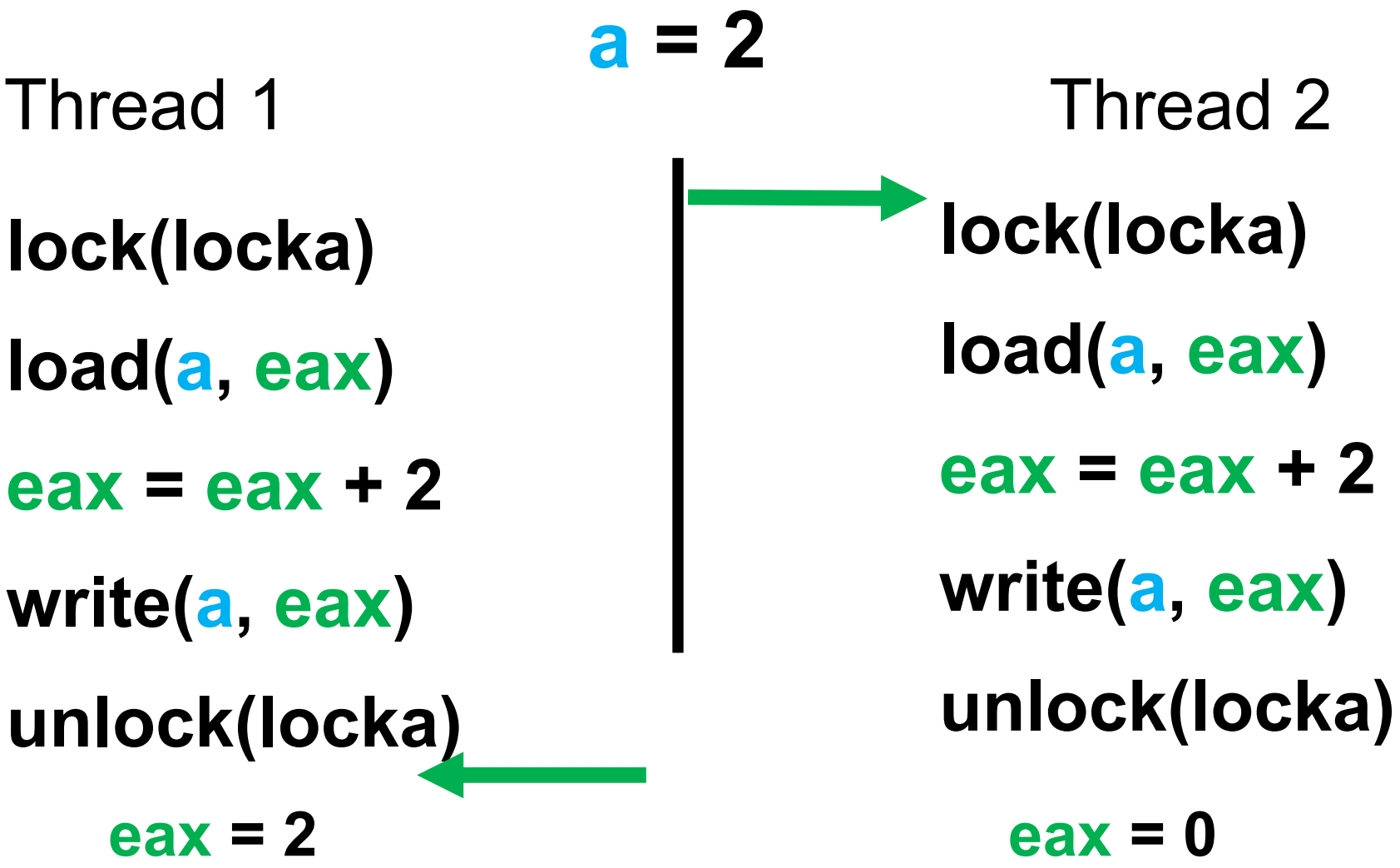
Race condition – soluție

OK

a = 0

Thread 1	Thread 2
lock(locka)	lock(locka)
load(a, eax)	load(a, eax)
eax = eax + 2	eax = eax + 2
write(a, eax)	write(a, eax)
unlock(locka)	unlock(locka)
eax = 2	eax = 0

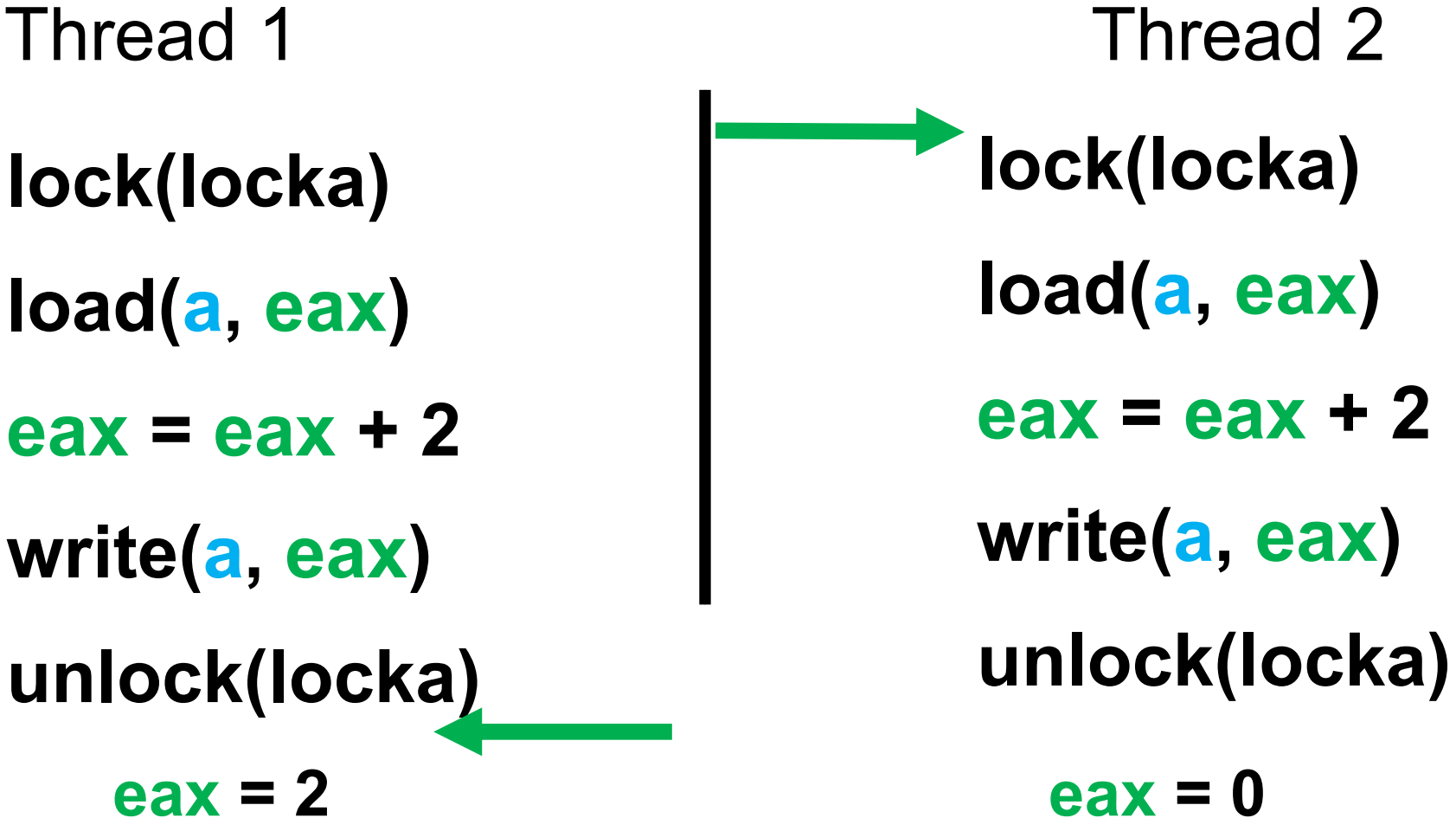
Race condition – soluție



Race condition – soluție

OK

a = 2



Race condition – soluție

a = 4

Thread 1	Thread 2
lock(locka)	lock(locka)
load(a, eax)	load(a, eax)
eax = eax + 2	eax = eax + 2
write(a, eax)	write(a, eax)
unlock(locka)	unlock(locka)
eax = 2	eax = 4

Race condition – soluție

OK

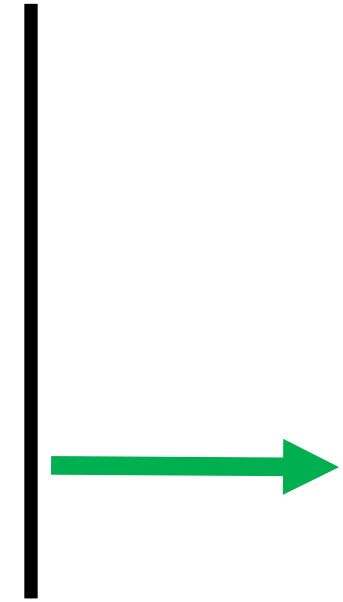
a = 4

Thread 1	Thread 2
lock(locka)	lock(locka)
load(a, eax)	load(a, eax)
eax = eax + 2	eax = eax + 2
write(a, eax)	write(a, eax)
unlock(locka)	unlock(locka)
eax = 2	eax = 4

Race condition – soluție

a = 0

Thread 1	Thread 2
lock(locka)	**lock(locka)**
load(a, eax)	**load(a, eax)**
eax = eax + 2	**eax = eax + 2**
write(a, eax)	**write(a, eax)**
unlock(locka)	**unlock(locka)**
eax = 0	**eax = 2**



Race condition – soluție

